PROJET RÉALISÉ POUR

LA PRÉDICTION D'OFFRES D'EMPLOIS

RAPPORT DE GROUPE EN SCIENCES DES DONNÉES 4

AIT ALI Lydia, BA Kardiatou, BOUHADJADJ Naim, GATEKA Diella Joannie .

Département MIASHS, UFR 6 Informatique, Mathématique et Statistique
Université Paul Valéry, Montpellier 3

Avril 2026

Soumis comme contribution
pour le cours Science des données 4

Déclaration de non plagiat

Nous déclarons que ce rapport est le fruit de notre seul travail, à part lorsque cela est indiqué explicitement.

Nous acceptons que la personne évaluant ce rapport puisse, pour les besoins de cette évaluation:

- la reproduire et en fournir une copie à un autre membre de l'université; et/ou,
- en communiquer une copie à un service en ligne de détection de plagiat (qui pourra en retenir une copie pour les besoins d'évaluation future).

Nous certifions que nous avons lu et compris les règles ci-dessus.

En signant cette déclaration, nous acceptons ce qui précède.

Signature: AIT ALI Lydia Date: 12/04/2026

Signature: BA Kardiatou Date: 12/04/2026

Signature: BOUHADJADJ Naim Date: 12/04/2026

Signature: GATEKA Diella Joannie Date: 12/04/2026

Remerciements

Nous adressons nos plus sincères remerciements à nos encadrantes pédagogiques, Mme Bringay, Mme Patel et Mr Michel-Picque, pour leurs précieux conseils, leur soutien constant et leur disponibilité tout au long de la réalisation de notre projet. Leur expertise a été essentielle pour le bon déroulement et le développement de notre travail.
Nous leur sommes profondément reconnaissantes pour le temps et l'énergie qu'ils ont consacrés à notre accompagnement, ainsi que pour la passion avec laquelle ils nous ont transmis leurs connaissances

Résumé

Le salaire est une question importante dans le monde du travail, car il peut varier selon plusieurs facteurs. On peut par exemple se demander si la région, le secteur d'activité, la taille de l'entreprise, le niveau de compétences ou encore l'usage de l'intelligence artificielle dans un métier ont un impact sur la rémunération. De la même manière, le risque d'automatisation peut aussi jouer un rôle dans les écarts de salaire observés entre différents emplois.

Pourtant, même si ces questions sont fréquentes, il n'est pas toujours facile d'y répondre de manière claire. Les données existent, mais elles ne sont pas toujours simples à comprendre ni à exploiter directement. Il peut donc être utile de mettre en place un outil capable à la fois d'estimer un salaire et de mieux montrer les facteurs qui influencent cette estimation.

C'est dans ce cadre que s'inscrit notre projet. Notre site web a pour objectif de prédire le salaire à partir de plusieurs caractéristiques liées à une offre d'emploi. Il s'appuie sur une base de données contenant des informations comme la région, le secteur d'activité, la taille de l'entreprise, le niveau de séniorité, l'intensité de l'IA, le métier ou encore le risque d'automatisation. La fonctionnalité principale du site est de permettre à l'utilisateur de renseigner ces différentes informations afin d'obtenir une estimation du salaire correspondant.

Pour construire cet outil, plusieurs modèles de machine learning ont été entraînés et évalués sur notre jeu de données. Cette étape nous a permis de comparer leurs performances et de retenir le modèle le plus adapté à notre objectif. Parmi les modèles testés, le Gradient Boosting est celui qui a donné les meilleurs résultats. C'est donc ce modèle qui a été choisi pour être intégré à notre site web afin de fournir les prédictions finales.

Au-delà de la prédiction, ce projet nous a également permis d'identifier les variables qui influencent le plus le salaire et de mieux comprendre les écarts de rémunération entre différents profils d'emploi. Il nous a ainsi amenés à travailler sur plusieurs étapes complémentaires, allant du nettoyage des données à l'exploration statistique, puis à la modélisation et à l'intégration dans une application web.

Table des matières

Chapitre 1 Introduction 1

Chapitre 2 Gestion de projet 2
- 2.1 Introduction 2
- 2.2 Collaboration avec Git 2
- 2.3 Contribution 2
- 2.4 Conclusions 3

Chapitre 3 Base de données 4
- 3.1 Introduction 4
- 3.2 Nettoyage des données 4
- 3.3 Modèle conceptuel des données 5
- 3.4 Modèle organisationnel des données 5
- 3.5 Conclusions 6

Chapitre 4 Exploration des données 7
- 4.1 Analyse 7
- 4.2 Quelques visualisations 8

Chapitre 5 Machine learning 10
- 5.1 Introduction 10
- 5.2 Exploration des données pour la modélisation 10
- 5.3 Prédiction 11
  - 5.3.1 Régression linéaire 12
  - 5.3.2 Gradient Boosting 16
  - 5.3.3 Random forest 18
  - 5.3.4 KNN (K-Nearest Neighbors) 21
  - 5.3.5 Réseau de neurones 24
  - 5.3.6 Comparaison des modèles 27

Chapitre 6 Description technique et fonctionnelle 29
- 6.1 Introduction 29
- 6.2 Description technique 29
- 6.3 Description fonctionnelle 31
  - 6.3.1 Page d'accueil 31
  - 6.3.2 Page *À propos* 32
  - 6.3.3 Page de prédiction 33
  - 6.3.4 Page de comparaison 34
  - 6.3.5 Page d'exploration des données 35

6.3.6 Page de contact 37
6.3.7 Espace administrateur 38
6.3.8 Page d'historique des prédictions 39
6.4 Conclusions 40

Chapitre 7 Conclusions et perspectives 41
7.0.1 Contributions du groupe 41
7.0.2 Perspectives 41

Annexes 43

Chapitre 1

Introduction

Le salaire est une question importante dans le monde du travail, car il peut varier selon plusieurs facteurs, comme le secteur d'activité, l'expérience, la région ou encore l'utilisation de l'intelligence artificielle. De plus, certains métiers sont aujourd'hui plus exposés à l'automatisation, ce qui peut également avoir une influence sur les rémunérations.

Dans ce contexte, on peut se demander quels sont les éléments qui influencent réellement le salaire et dans quelle mesure il est possible de l'estimer à partir de certaines caractéristiques d'un emploi. Pour répondre à cette question, nous avons développé un site web capable de prédire un salaire à l'aide de méthodes de machine learning.

Ceci nous a ainsi conduits à travailler sur plusieurs étapes complémentaires, allant du nettoyage des données à leur exploration, puis à la modélisation et à l'intégration du modèle dans une application web.

Lien de la vidéo de soutenance :
https://www.youtube.com/watch?v=cpAHGd8Ny-8

Lien de démonstration du site web :
https://www.youtube.com/watch?v=iz_PRw1_i4U

Chapitre 2

Gestion de projet

2.1 Introduction

Le projet a été structuré en plusieurs phases : le nettoyage et l'exploration des données, la conception de la base de données, la modélisation en machine learning, puis l'intégration du modèle dans un site web. Cette organisation nous a permis de travailler de manière progressive et cohérente.

Enfin, une répartition des tâches a été mise en place entre les membres du groupe afin d'assurer une bonne coordination et d'optimiser l'avancement du projet.

2.2 Collaboration avec Git

Git nous a permis de travailler de manière collaborative en partageant progressivement les différentes avancées liées à la création des pages du site web. Chaque membre du groupe pouvait ainsi suivre l'évolution du travail des autres et accéder aux versions les plus récentes du projet.

Le dépôt GitHub a également servi d'espace central de sauvegarde, ce qui nous a permis de conserver l'historique du projet et de sécuriser notre travail tout au long du développement.

Pour limiter les conflits de fusion, nous avons utilisé des sous-branches afin de tester certaines fonctionnalités avant d'intégrer une version stable au projet principal. Cette organisation nous a permis de mieux contrôler les modifications apportées au site.

Nous avons rencontré quelques conflits de *merge*, mais ils sont restés limités. Ils apparaissaient surtout lorsqu'un membre essayait de *push* ses modifications alors qu'un autre avait déjà modifié la même partie du projet. Dans ce cas, nous avons dû mettre à jour le dépôt local, comparer les changements et résoudre manuellement les différences.

Lien du Github :https://github.com/joannie15/Projet_science_de_donn-es_4

2.3 Contribution

Les figures suivantes présentent la répartition des tâches entre les membres du groupe:

Tâche	Lydia	Joannie	Kardiatou	Naim
Nettoyage et exploration des données			100%	
Importation des données sur Sql		100%		
MCD/MOD		100%		
Machine learning			100%	
Rapport : google doc		55%	45%	
Rapport : latex			100%	
Support Canva	100%			
Vidéo de présentation du site				100%

Figure 2.1: Contribution des tâches du projet

Site	Lydia	Joannie	Kardiatou	Naim
Admin/index.php			100%	
README.md			100%	
Cache/categories.json		100%		
ancien_site_avec_prediction_fonctionnel_code_different.zip				100%
apropos.php		100%		
comparaison.php			100%	
config.php				100%
contact.php		100%		
db.sql			100%	
exploration.php				100%
historique.php		100%		
index.php		100%		
prediction.php		100%		
style.css		100%		

Figure 2.2: Contribution des tâches des pages du site web

2.4 Conclusions

Ainsi, une bonne organisation et l'utilisation d'outils collaboratifs ont permis d'assurer une coordination efficace et le bon déroulement du projet.

Chapitre 3

Base de données

3.1 Introduction

Source de données: https://www.kaggle.com/datasets/sarcasmos/ai-society (format csv,22 colonnes et 5000 lignes).

Ce jeu de données regroupe des informations relatives à l'impact de l'intelligence artificielle sur différentes offres d'emploi. Il contient des variables portant sur les salaires, les intitulés de poste, les secteurs d'activité, la localisation géographique, le niveau de compétences requis, ainsi que la présence de l'intelligence artificielle dans les offres. Chaque ligne correspond à une offre d'emploi unique, ce qui permet d'analyser plusieurs dimensions, telles que l'adoption de l'intelligence artificielle, les compétences recherchées, l'évolution des salaires, le risque d'automatisation ou encore le niveau de maturité de l'IA selon les secteurs d'activité.

3.2 Nettoyage des données

Avant de concevoir la base de données et de réaliser l'exploration des données, nous avons procédé à une phase de nettoyage du jeu de données. Nous avons d'abord supprimé les colonnes considérées comme non pertinentes pour notre étude, notamment « **ai_skills** » en raison du nombre important de valeurs manquantes, ainsi que « **job_description_embedding_cluster** » que nous avons jugé non nécessaire pour notre projet. Nous avons également renommé certaines colonnes afin d'en faciliter la compréhension et le repérage, ou parce que leurs intitulés étaient trop longs.

Avant toute exploitation des données, nous avons vérifié l'absence de valeurs manquantes et de doublons. Dans une logique de structuration rigoureuse de la base de données, nous avons ensuite créé, à l'aide d'Excel, plusieurs fichiers CSV correspondant aux différentes tables, chacun comportant des identifiants uniques.

Enfin, nous avons utilisé Python pour générer les clés étrangères nécessaires à la mise en place des relations entre les différentes tables. Nous avons utilisé la bibliothèque *pandas* afin d'associer les identifiants uniques aux données de la table principale (*offre*).

Pour chaque variable descriptive (poste, entreprise, pays, région, etc.), nous avons réalisé une jointure entre la table principale et les tables de référence correspondantes. Cette opération permet de récupérer les identifiants (clés primaires) des tables secondaires et de les intégrer dans la table *offre* sous forme de clés étrangères

Ces choix ont permis de réduire la complexité du jeu de données tout en conservant les variables pertinentes pour l'analyse.

3.3 Modèle conceptuel des données

Le MCD (Modèle Conceptuel de Données) constitue une représentation abstraite de la base de données. Il permet d'identifier les entités, leurs attributs ainsi que les relations qui les lient, indépendamment de toute implémentation technique.

Une représentation graphique du MCD, réalisée à l'aide du logiciel *Mocodo* (https://www.mocodo.net/), est présentée ci-dessous afin d'illustrer cette modélisation conceptuelle.

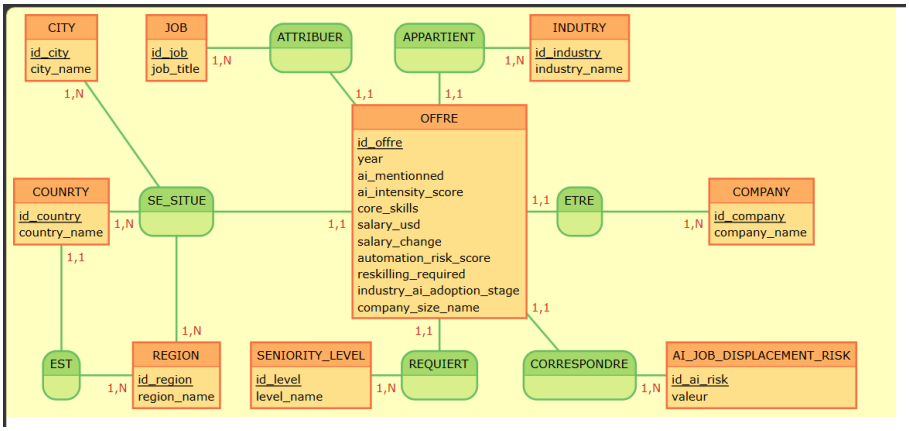

Figure 3.1: Modèle conceptuel des données (MCD)

Dans notre modèle, l'entité centrale est **OFFRE**. Elle regroupe les informations principales relatives aux offres d'emploi, telles que le salaire, l'année ou encore le score d'intensité de l'intelligence artificielle. Cette entité est reliée aux différentes autres tables, qui permettent de décrire plus précisément le poste, l'entreprise, la localisation ainsi que certaines caractéristiques associées à l'offre, telles que le risque de remplacement par l'intelligence artificielle et le niveau hiérarchique.

3.4 Modèle organisationnel des données

À partir du MCD, nous avons ensuite réalisé le MOD, représenté ici par une image générée avec le Designer de phpMyAdmin. Le MOD montre de manière concrète comment les données sont organisées dans MySQL, notamment à travers les clés primaires, les clés étrangères ainsi que les différents attributs de chaque table, comme illustré dans la Figure 3.2.

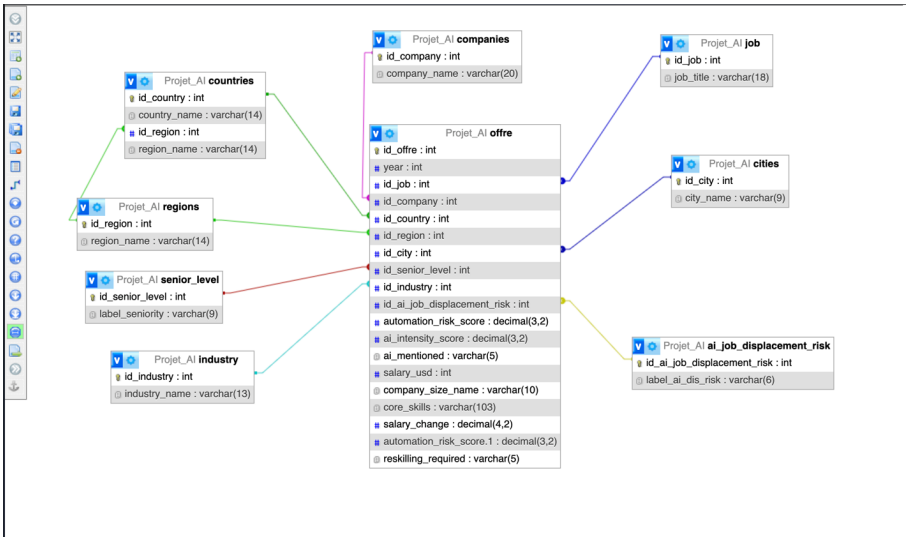

Figure 3.2: Modèle organisationnel des données (MOD)

3.5 Conclusions

La conception du MCD et du MOD nous a permis d'obtenir une vision claire, structurée et cohérente de l'organisation des données, en mettant en évidence les différentes entités ainsi que les relations existant entre les tables. Cette étape a été essentielle pour garantir une bonne compréhension globale de la base de données avant son implémentation.

La mise en place complète de la base de données nous a pris environ trois jours. Cette durée inclut la conception des modèles, la structuration des tables ainsi que l'intégration des relations entre les différentes entités.

Nous n'avons pas rencontré de difficultés majeures lors de l'exportation de la base de données. Cependant, nous avons constaté que lors de son importation par d'autres utilisateurs, certaines relations entre les tables, pourtant correctement définies, ne sont pas toujours conservées dans l'environnement phpMyAdmin. Ce point constitue une limite technique liée à l'outil utilisé plutôt qu'à la conception de la base elle-même.

Chapitre 4

Exploration des données

4.1 Analyse

Après l'étape de nettoyage des données, une phase d'exploration a été menée afin de mieux comprendre la structure du jeu de données et d'identifier les relations entre les différentes variables. Cette étape est essentielle, car elle permet de mettre en évidence les tendances générales, de détecter d'éventuelles anomalies, ainsi que d'identifier les variables les plus pertinentes pour la partie machine learning.

Pour cela, plusieurs types d'analyses ont été réalisés. Les analyses univariées ont permis d'étudier chaque variable individuellement, afin d'observer leur répartition et de mieux comprendre leur contenu. Les analyses bivariées ont ensuite été utilisées pour examiner les relations entre deux variables, en particulier entre les variables explicatives et la variable cible. Enfin, les analyses multivariées ont permis d'obtenir une vision plus globale des interactions entre plusieurs variables simultanément.

Une attention particulière a été portée à la variable *salary_usd*, puisqu'il s'agit de la variable que nous cherchons à prédire. L'objectif de cette exploration était donc de mieux comprendre comment le salaire varie selon différents critères, tels que la région, le secteur d'activité, le niveau hiérarchique, la taille de l'entreprise, l'intensité de l'intelligence artificielle ou encore le risque d'automatisation.

Cette phase exploratoire a également permis de mettre en évidence certaines caractéristiques importantes du jeu de données, notamment la présence de valeurs extrêmes sur les salaires, ainsi que des relations plus ou moins marquées entre certaines variables et la rémunération. Ces premiers constats ont été déterminants pour orienter la suite du projet, notamment dans le choix des variables à conserver et dans la mise en place des modèles de prédiction.

4.2 Quelques visualisations

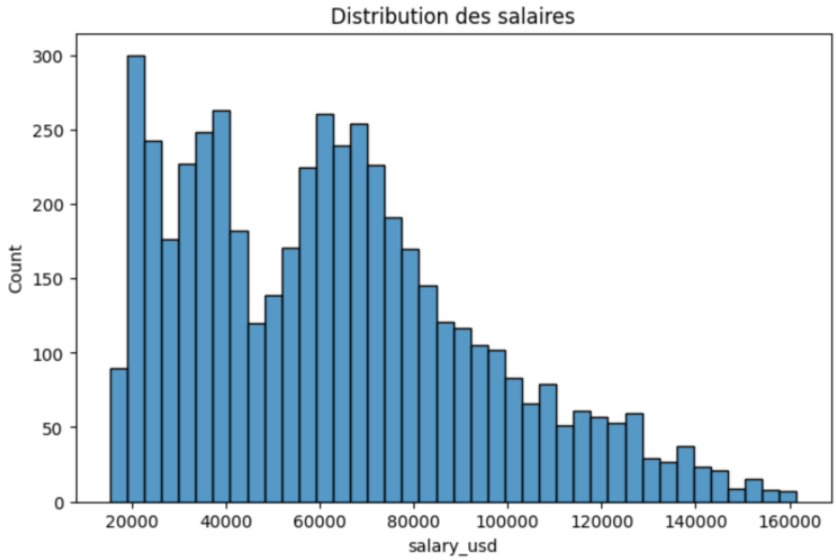

Figure 4.1: Distribution des salaires

La Figure 4.1 montre la distribution des salaires, qui présente une asymétrie à droite. La majorité des observations se concentre entre 40 000 et 80 000 USD, tandis qu'un nombre plus limité d'individus perçoit des salaires nettement plus élevés.

Cette distribution met en évidence une forte inégalité des rémunérations, caractérisée par la présence de valeurs extrêmes élevées. Ces valeurs peuvent correspondre à des profils très qualifiés ou à des postes spécifiques fortement rémunérés, ce qui contribue à étirer la distribution vers la droite.

Figure 4.2: Box-plot de l'évolution des salaires au fil des années et selon le niveau d'expérience

La Figure 4.2 met en évidence une hausse globale des salaires au fil des années, quel que soit le niveau d'expérience. Il apparaît également que les profils les plus expérimentés, tels que les niveaux Senior, Lead ou Executive, bénéficient de rémunérations plus élevées que les profils Junior ou Intern.

Par ailleurs, l'élargissement progressif des boîtes et des moustaches au cours des années récentes traduit une augmentation de la dispersion des salaires. Cela suggère une plus grande variabilité des rémunérations, possiblement liée à une diversification des profils, des postes ou des secteurs d'activité.

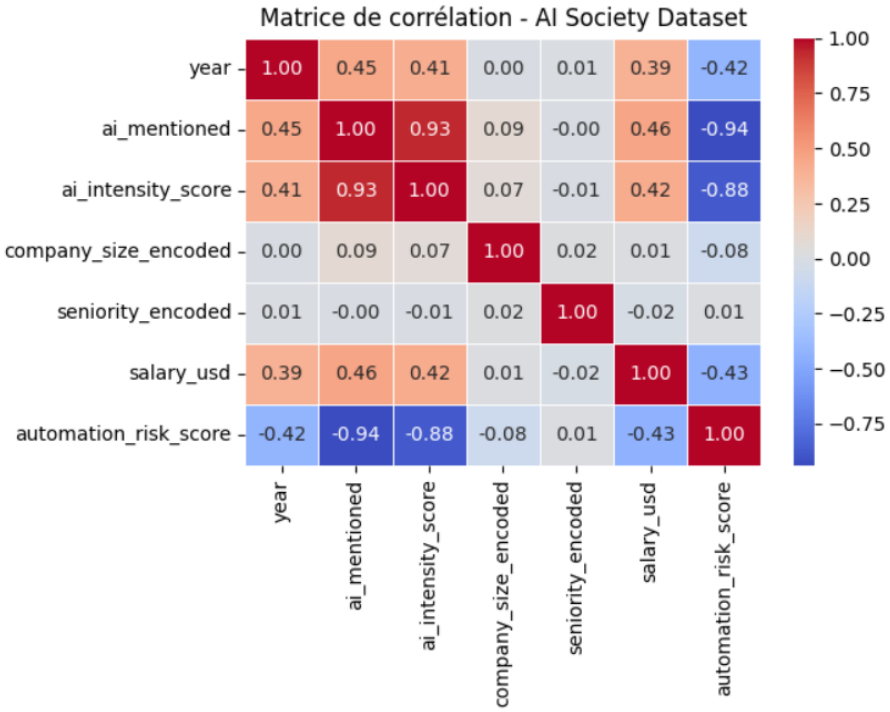

Figure 4.3: Matrice de corrélation

La Figure 4.3 met en évidence les corrélations entre les différentes variables du jeu de données. On observe que le salaire est positivement corrélé avec les variables liées à l'intelligence artificielle, en particulier *ai_mentioned* et *ai_intensity_score*, ainsi qu'avec l'année.

Cela suggère que les emplois davantage orientés vers l'intelligence artificielle, ainsi que les périodes les plus récentes, sont associés à des rémunérations plus élevées. À l'inverse, le risque d'automatisation est négativement corrélé au salaire, ce qui indique que les métiers les moins automatisables sont globalement mieux rémunérés.

Ces résultats confirment l'importance de certaines variables explicatives dans la variation du salaire et justifient leur utilisation dans les modèles de prédiction.

Chapitre 5

Machine learning

5.1 Introduction

Notre objectif est de prédire le salaire d'une offre d'emploi à partir de plusieurs variables explicatives, telles que l'année, la région, le secteur d'activité, le niveau de risque d'automatisation, la taille de l'entreprise, l'intensité de l'intelligence artificielle, l'intitulé du poste ainsi que le niveau de compétence associé à l'offre.

5.2 Exploration des données pour la modélisation

Dans le cadre de cette partie consacrée au machine learning, une attention particulière a été portée à la qualité et à la pertinence des données utilisées pour la prédiction. L'analyse du jeu de données a permis de mettre en évidence ses principales caractéristiques, notamment sa dimension, sa structure et la qualité générale des informations disponibles.

Cette vérification a confirmé l'absence de valeurs manquantes et de données mal formatées, constituant ainsi une base fiable pour la suite de l'étude.

Une réflexion a également été menée sur le choix des variables explicatives. Un point de vigilance important concernait la fuite de cible (*target leakage*), c'est-à-dire la présence de variables contenant, directement ou indirectement, des informations trop liées à la variable à prédire. Il était donc essentiel de ne conserver que les variables réellement utiles à la modélisation, afin d'obtenir des prédictions cohérentes et fiables.

La matrice de corrélation présentée dans la Figure 4.3 montre par exemple que certaines variables, comme *job_ id*, n'ont pas d'influence significative sur le salaire, leur corrélation étant proche de zéro. À l'inverse, la variable *salary_ change* semble dériver directement de la variable cible, ce qui aurait pu introduire un biais important dans les résultats du modèle.

Pour ces raisons, ces variables ont été exclues du processus de modélisation afin de garantir la fiabilité et la pertinence des prédictions obtenues.

Dans cette perspective, les visualisations graphiques ont joué un rôle important, car elles ont permis de mieux comprendre les relations entre les variables et d'orienter le choix des variables explicatives à retenir. Elles ont ainsi contribué à une meilleure interprétation du jeu de données avant l'entraînement des modèles.

Enfin, la question des valeurs aberrantes a également été examinée. L'analyse a montré que les salaires particulièrement élevés ne représentaient que 0,6 % de

l'ensemble des observations. Leur suppression n'a donc pas été jugée nécessaire, afin de préserver la représentativité du jeu de données.

```
Q1 = df["salary_usd"].quantile(0.25)
Q3 = df["salary_usd"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print(lower)

print(upper)

outliers = df[(df["salary_usd"] < lower) | (df["salary_usd"] > upper)]

print("Nombre d'outliers :", len(outliers))
print("Pourcentage :", len(outliers) / len(df) * 100)
print(outliers[["salary_usd", "job_title", "region", "industry","seniority_level","ai_intensity_score"]])
```

```
-31278.875
149666.125
Nombre d'outliers : 30
Pourcentage : 0.6
```

Figure 5.1: Code utilisé pour identifier les valeurs aberrantes

La méthode de l'IQR, présentée dans le code de la Figure 5.1, permet d'identifier les valeurs atypiques en définissant un intervalle de valeurs dites « normales » à partir des quartiles Q1 et Q3.

La borne inférieure obtenue est négative. Cela ne constitue pas une erreur : ce résultat s'explique par une dispersion importante des salaires, qui étend l'intervalle théorique en dessous de 0. Cependant, cette borne n'est pas pertinente dans ce contexte, puisqu'un salaire ne peut pas être négatif. Cela indique donc l'absence de valeurs anormalement faibles dans le jeu de données.

5.3 Prédiction

La bibliothèque *Scikit-learn* de Python a été utilisée pour mettre en place la prédiction des salaires des différentes offres d'emploi. Les variables explicatives retenues correspondent à plusieurs caractéristiques des offres, telles que l'année, la région, le secteur d'activité, le niveau de séniorité, la taille de l'entreprise ou encore l'intensité de l'intelligence artificielle.

Dans un premier temps, les données ont été divisées en deux ensembles : un jeu d'entraînement (80 % des données), permettant d'entraîner les modèles, et un jeu de test (20 % des données), utilisé pour évaluer leurs performances. Cette séparation permet d'évaluer les modèles sur des données qu'ils n'ont pas vues pendant l'entraînement, afin de mesurer leur capacité de généralisation.

L'implémentation de cette étape est présentée dans la Figure 5.2.

```python
from sklearn.model_selection import train_test_split, cross_val_score
import pandas as pd

# Variable cible
y = df["salary_usd"]

# Variables explicatives
X = df[[
    "year",
    "region",
    "industry",
    "automation_risk_score",
    "company_size",
    "seniority_level",
    "ai_intensity_score",
    "job_title"
]]
# Encodage des variables qualitatives
X = pd.get_dummies(X)

# Découpage train / test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Figure 5.2: Préparation des variables explicatives et séparation des données

Nous avons testé cinq modèles de régression : la régression linéaire, le Gradient Boosting, le K-Nearest Neighbors (KNN), le Random Forest et le réseau de neurones. L'objectif est de comparer différentes approches, allant des modèles les plus simples aux modèles plus complexes, afin d'identifier celui qui s'adapte le mieux à notre problème de prédiction des salaires.

5.3.1 Régression linéaire

La régression linéaire est un modèle prédictif qui établit une relation linéaire entre une variable dépendante et plusieurs variables explicatives. Elle consiste à estimer des coefficients en minimisant la somme des carrés des écarts entre les valeurs observées et les valeurs prédites, sous certaines hypothèses. Une fois ces coefficients estimés, la prédiction est obtenue en combinant linéairement les différentes variables explicatives.

Pour évaluer les performances des modèles, nous avons utilisé plusieurs métriques complémentaires : le RMSE, le MAE et le coefficient de détermination R^2. Chacune apporte une information différente sur la qualité des prédictions.

Le MAE (*Mean Absolute Error*) correspond à l'erreur absolue moyenne entre les valeurs réelles et les valeurs prédites, sans tenir compte du sens de l'erreur. Autrement dit, il mesure en moyenne de combien le modèle se trompe. Par exemple, un MAE de 5 000 signifie que les prédictions s'écartent en moyenne de 5 000 dollars des valeurs réelles. Cette métrique est simple à interpréter et donne une vision globale de l'erreur.

Le RMSE (*Root Mean Squared Error*) est proche du MAE, mais il accorde davantage d'importance aux erreurs les plus élevées. En effet, les écarts sont mis au carré avant d'être moyennés, ce qui pénalise fortement les grandes différences entre les valeurs réelles et les valeurs prédites. Cette métrique est particulièrement utile lorsque l'on souhaite éviter les erreurs importantes, par exemple dans le cas de salaires très élevés.

Enfin, le coefficient de détermination R^2 permet d'évaluer la qualité globale du modèle. Il indique la proportion de la variabilité de la variable cible, ici le salaire, qui est expliquée par les variables explicatives. Sa valeur est généralement comprise entre 0 et 1 : plus elle est proche de 1, plus le modèle est performant. Par exemple, un R^2 de 0,96 signifie que 96 % des variations du salaire sont expliquées par le modèle.

La Figure 5.3 présente les résultats de prédiction du salaire obtenus avec le modèle de régression linéaire.

```python
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


model_lr = LinearRegression()

# Entraînement
model_lr.fit(X_train, y_train)

# Prédiction
y_pred_lr = model_lr.predict(X_test)

mse_lr = mean_squared_error(y_test, y_pred_lr)
rmse_lr = np.sqrt(mse_lr)
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print("\nÉvaluation finale sur le test")
print("RMSE:", rmse_lr)
print("MAE:", mae_lr)
print("R2:", r2_lr)
```

```
Évaluation finale sur le test
RMSE: 9496.938006651528
MAE: 7406.0019355824015
R2: 0.913484298223104
```

Figure 5.3: Prédictions du salaire avec la régression linéaire

Dans le cas de la régression linéaire, le RMSE est de 9 496,94, ce qui signifie que le modèle commet une erreur d'environ 9 500 dollars sur la prédiction du salaire.

Le MAE est d'environ 7 400 dollars, ce qui indique que le modèle se trompe en moyenne de 7 400 dollars par prédiction.

Le coefficient de détermination R^2 est de 0,91, ce qui montre que le modèle explique environ 91 % de la variabilité du salaire.

Afin d'obtenir une évaluation plus fiable, nous avons également utilisé une validation croisée à 5 plis. Cette méthode consiste à diviser les données d'apprentissage en plusieurs sous-ensembles, puis à entraîner et évaluer le modèle sur différentes partitions. Elle permet ainsi de limiter l'influence d'un découpage aléatoire unique et d'obtenir une estimation plus robuste des performances.

```python
# Validation croisée

scores = cross_val_score(model_lr, X, y, cv=5, scoring="r2")

print("Scores R2 (validation croisée) :", scores)
print("Score moyen CV :", scores.mean())
```

```
Scores R2 (validation croisée) : [0.9076388  0.90923966 0.91523879 0.91616056 0.90923839]
Score moyen CV : 0.9115032386442763
```

Figure 5.4: Résultats de la validation croisée pour la régression linéaire

La validation croisée donne des scores R^2 compris entre 0,90 et 0,91, avec une moyenne d'environ 0,91. Ces résultats sont très proches entre les différents plis, ce qui montre que le modèle est stable et ne dépend pas fortement du découpage des données.

Le modèle présente donc une bonne capacité de généralisation, avec des performances cohérentes entre le jeu d'entraînement, le jeu de test et la validation croisée.

Cependant, malgré ces bons résultats, la régression linéaire reste limitée dans sa capacité à modéliser des relations complexes entre les variables explicatives et le salaire, ce qui justifie l'utilisation de modèles non linéaires plus avancés dans la suite de l'étude.

Analyse des résidus

Afin d'évaluer la qualité des prédictions du modèle, nous avons mesuré l'écart entre les valeurs réelles et les valeurs prédites. Pour cela, nous avons analysé les résidus, c'est-à-dire la différence entre les valeurs observées et celles estimées par le modèle. Cette analyse repose notamment sur l'étude de leur distribution ainsi que sur leur représentation en fonction des valeurs prédites. Cette dernière permet de vérifier si les erreurs sont réparties de manière aléatoire ou si elles présentent une structure particulière.

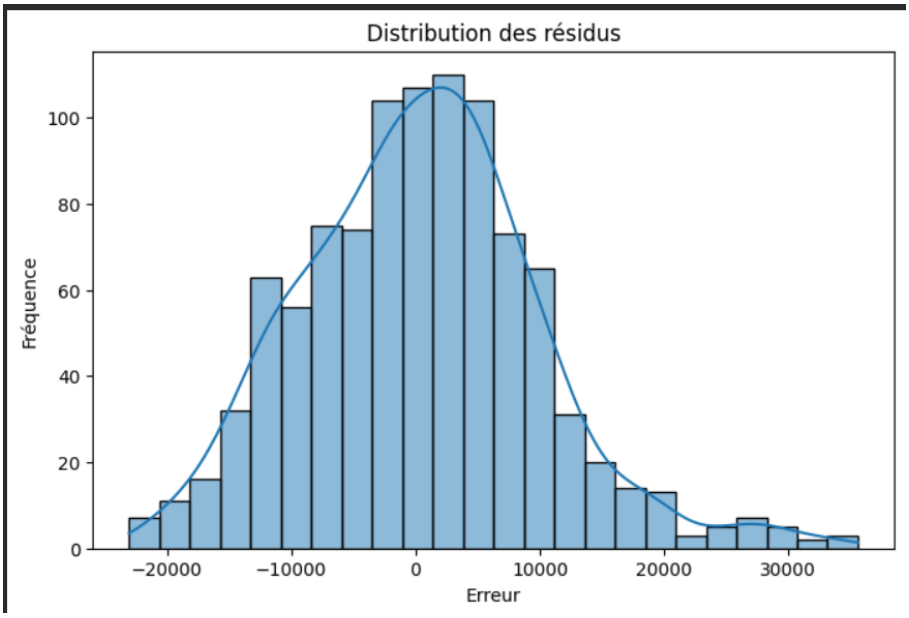

Figure 5.5: Distribution des résidus de la régression linéaire

La Figure 5.5 montre que la distribution des résidus est globalement centrée autour de zéro et présente une forme proche d'une loi normale, ce qui valide en partie les hypothèses de la régression linéaire.

Cependant, une légère asymétrie vers la droite est observée, indiquant que le modèle a tendance à sous-estimer les salaires les plus élevés.

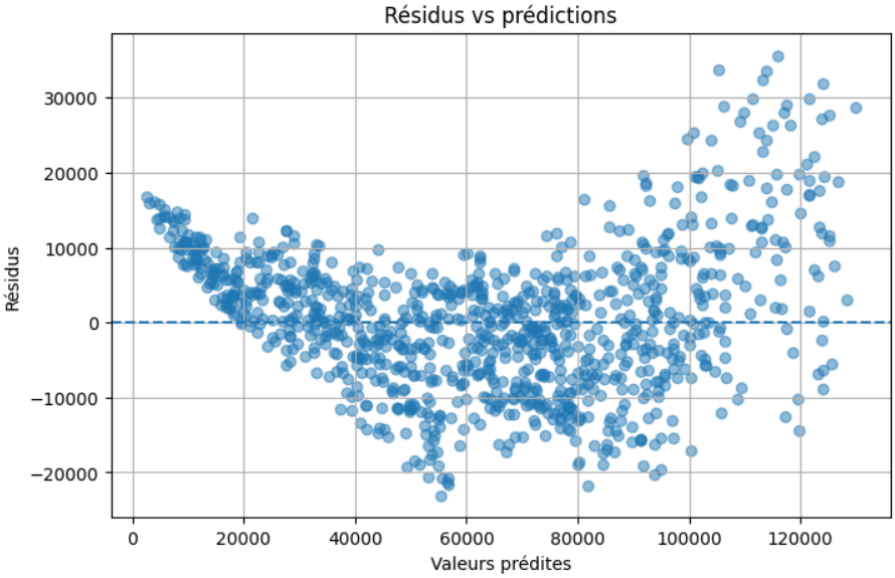

Figure 5.6: Résidus en fonction des valeurs prédites

La Figure 5.6 met en évidence que les erreurs ne sont pas totalement aléatoires. On observe notamment une forme légèrement courbée, ce qui suggère que la relation entre les variables explicatives et le salaire n'est pas strictement linéaire.

Par ailleurs, la dispersion des résidus augmente pour les salaires élevés, indiquant que le modèle est moins précis pour ces valeurs. Ce phénomène traduit une difficulté du modèle linéaire à capturer les variations importantes des salaires élevés.

Ces observations confirment que, bien que la régression linéaire fournisse des résultats satisfaisants, elle présente certaines limites face à la complexité des relations présentes dans les données.

5.3.2 Gradient Boosting

Le Gradient Boosting Regressor est un modèle d'ensemble reposant sur la combinaison de plusieurs arbres de décision faibles. Il construit les arbres de manière séquentielle : chaque nouvel arbre est entraîné pour corriger les erreurs commises par les modèles précédents.

Le principe repose sur une optimisation progressive de la fonction de perte, généralement par descente de gradient. À chaque itération, le modèle s'ajuste en mettant davantage l'accent sur les observations mal prédites, ce qui permet d'améliorer progressivement la qualité des prédictions.

La prédiction finale est obtenue en combinant l'ensemble des arbres construits, ce qui permet au modèle de capturer des relations complexes et non linéaires entre les variables explicatives et la variable cible.

Ce type de modèle est particulièrement adapté aux problèmes de prédiction complexes, où les relations entre les variables ne sont pas strictement linéaires.

```python
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

model_gb = GradientBoostingRegressor(random_state=42)

model_gb.fit(X_train, y_train)

y_pred_gb = model_gb.predict(X_test)

mse_gb = mean_squared_error(y_test, y_pred_gb)
rmse_gb = np.sqrt(mse_gb)
mae_gb = mean_absolute_error(y_test, y_pred_gb)
r2_gb = r2_score(y_test, y_pred_gb)

print("\n--- Évaluation finale ---")
print("RMSE:", rmse_gb)
print("MAE:", mae_gb)
print("R2:", r2_gb)
```

```
--- Évaluation finale ---
RMSE: 6395.021673470659
MAE: 4913.543776692184
R2: 0.9607705905696431
```

Figure 5.7: Prédictions du salaire avec le Gradient Boosting

La Figure 5.7 montre que le modèle de Gradient Boosting fournit des résultats très satisfaisants. Avec un coefficient de détermination R^2 proche de 0,96, il parvient à expliquer une grande partie de la variabilité du salaire.

De plus, les valeurs du RMSE et du MAE, respectivement de 6 400 dollars et 4 900 dollars, indiquent que les écarts entre les salaires réels et les salaires prédits restent globalement faibles. Cela confirme la bonne qualité de prédiction du modèle sur ce jeu de données.

Le code présenté ci-dessous correspond à la mise en œuvre de la validation croisée :

```
#Validation croisée
scores = cross_val_score(model_gb, X, y, cv=5, scoring="r2")

print("Scores R2 (CV) :", scores)
print("Score moyen CV :", scores.mean())
```

```
Scores R2 (CV) : [0.95693851 0.96054914 0.96175956 0.96192637 0.95970772]
Score moyen CV : 0.9601762595856693
```

Figure 5.8: Résultats de la validation croisée pour le Gradient Boosting

Les scores obtenus avec la validation croisée sont très homogènes, avec un R^2 moyen d'environ 0,96. Cela confirme la stabilité et la robustesse du modèle sur différents sous-ensembles de données.

Analyse des résidus

Le graphique ci-dessous représente la distribution des résidus.

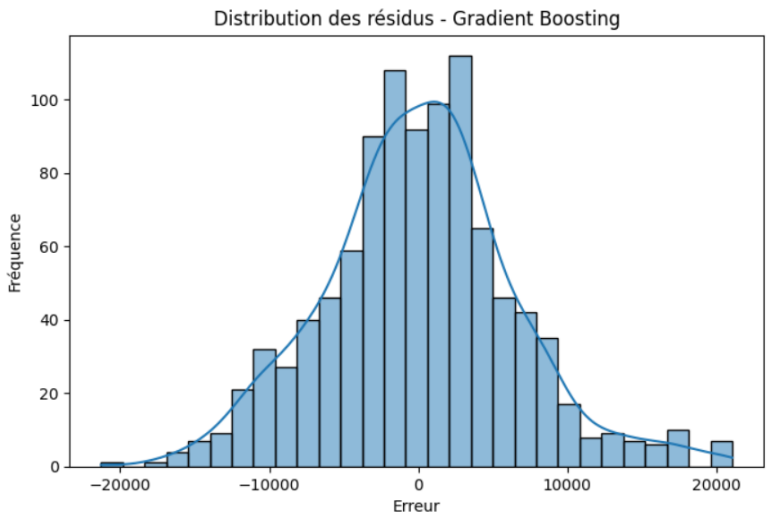

Figure 5.9: Distribution des résidus du Gradient Boosting

On observe que les résidus sont majoritairement concentrés autour de zéro, ce qui traduit une bonne qualité globale de prédiction. La distribution reste relativement équilibrée, même si quelques erreurs plus importantes subsistent aux extrémités. Cela suggère que le modèle prédit correctement la majorité des salaires, tout en rencontrant davantage de difficultés pour certains cas particuliers.

Nous avons également représenté les résidus en fonction des valeurs prédites, comme le montre la Figure 5.10.

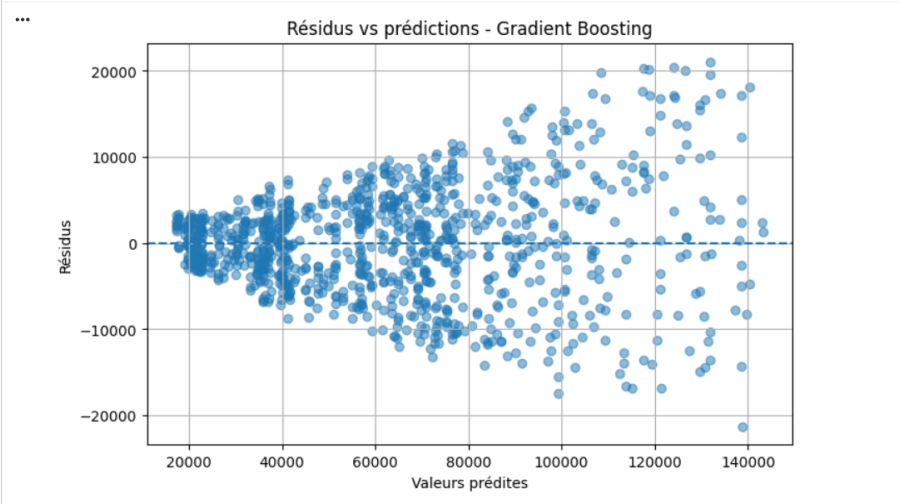

Figure 5.10: Résidus en fonction des valeurs prédites pour le Gradient Boosting

Le graphique des résidus en fonction des valeurs prédites montre que les erreurs sont globalement centrées autour de zéro, ce qui indique l'absence de biais global. On observe que le modèle est plutôt performant pour prédire les salaires les moins élevés. Cependant, la dispersion des résidus augmente avec les valeurs prédites, suggérant que le modèle est moins précis pour les salaires élevés. La présence de certaines erreurs importantes indique également que certaines observations restent difficiles à prédire.

5.3.3 Random forest

Le Random Forest Regressor est un modèle d'ensemble qui repose sur la construction de plusieurs arbres de décision. Chaque arbre est entraîné sur une partie aléatoire des données et des variables, puis leurs prédictions sont combinées pour obtenir un résultat final. Cela permet d'obtenir un modèle plus stable et généralement plus performant.

```python
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

model_rf = RandomForestRegressor(random_state=42)

model_rf.fit(X_train, y_train)

y_pred_rf = model_rf.predict(X_test)

mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
mae_rf = mean_absolute_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print("\n--- Évaluation finale ---")
print("RMSE:", rmse_rf)
print("MAE:", mae_rf)
print("R2:", r2_rf)
```

```
--- Évaluation finale ---
RMSE: 6795.105116271793
MAE: 5139.03551
R2: 0.9557085306945817
```

Figure 5.11: Prédictions du salaire avec le Random Forest

Le modèle de Random Forest présente de très bonnes performances, avec un R^2 d'environ 0,956, ce qui signifie qu'il explique près de 95,6 % de la variabilité du salaire. Cela indique que le modèle est capable de bien capturer les relations entre les variables explicatives et la variable cible. Les erreurs restent relativement faibles, avec un RMSE d'environ 6 795 dollars et un MAE d'environ 5 139 dollars. Cela signifie que, en moyenne, les prédictions du modèle s'écartent de quelques milliers de dollars des valeurs réelles, ce qui reste acceptable au regard des niveaux de salaire observés dans le jeu de données.

Le code ci-dessous correspond à la validation croisée:

```python
scores = cross_val_score(model_rf, X, y, cv=5, scoring="r2")
print("Score R2 (CV) :", scores)
print("Score moyen CV :", scores.mean())
```

```
Score R2 (CV) : [0.95223741 0.95723061 0.95730409 0.95747909 0.95613754]
Score moyen CV : 0.9560777498889358
```

Figure 5.12: Résultats de la validation croisée pour le Random Forest

La validation croisée donne des scores R^2 compris entre 0,952 et 0,957, avec une moyenne d'environ 0,956. Les résultats sont proches entre les différents plis, ce qui montre que le modèle est stable et peu sensible au découpage des données.

Analyse des résidus

La figure ci-dessous présente la distribution des résidus associée au modèle Random Forest, permettant d'analyser la répartition des erreurs de prédiction.

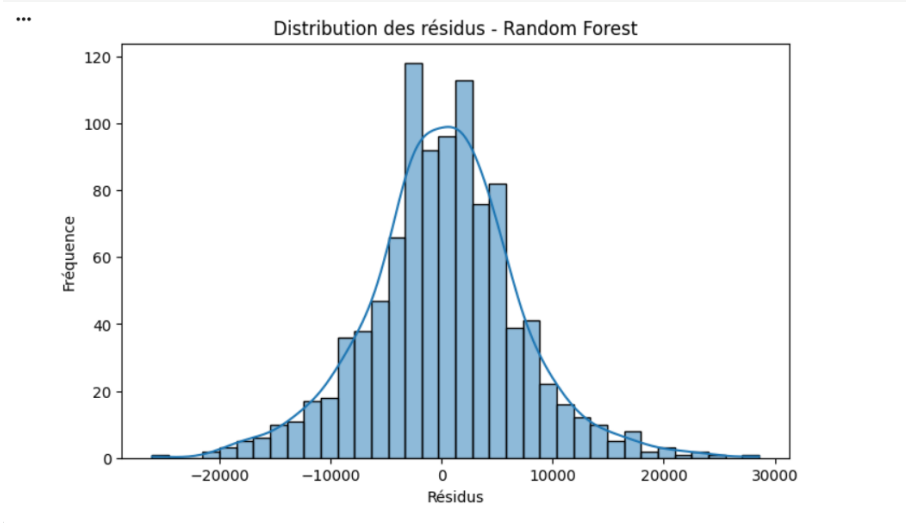

Figure 5.13: Distribution des résidus pour le Random Forest

On observe que la distribution des résidus est globalement centrée autour de zéro et présente une forme proche d'une loi normale, ce qui indique que les erreurs sont réparties de manière relativement aléatoire et que le modèle ne présente pas de biais systématique.

Toutefois, l'étendue relativement importante des résidus, ainsi que la présence de valeurs extrêmes, montrent que certaines prédictions restent moins précises, en particulier pour des observations atypiques.

Nous avons également représenté les résidus en fonction des valeurs prédites, comme illustré dans la figure ci-dessous :

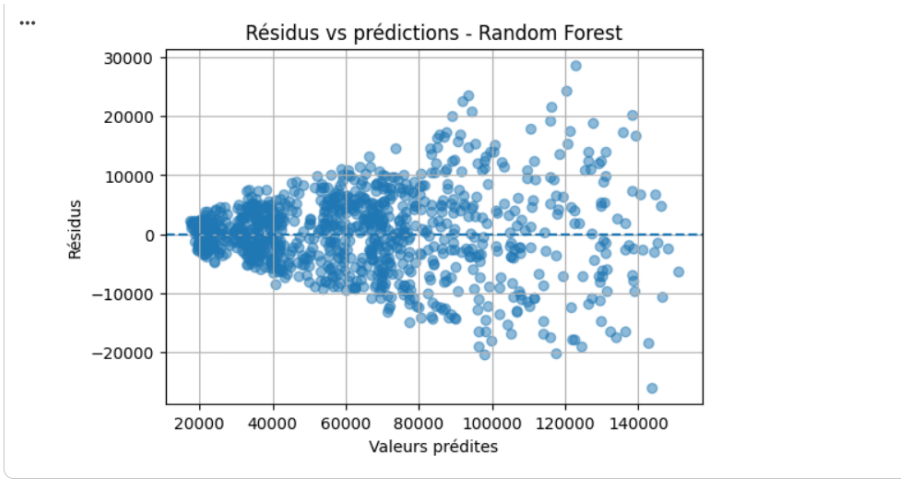

Figure 5.14: Résidus en fonction des valeurs prédites pour le Random Forest

Le graphique des résidus en fonction des valeurs prédites montre que les erreurs sont globalement centrées autour de zéro, ce qui indique l'absence de biais systématique du modèle Random Forest.

Toutefois, on observe une augmentation de la dispersion des résidus pour les valeurs prédites élevées. Cela signifie que le modèle est moins précis pour les grandes valeurs. Enfin, la présence de valeurs extrêmes indique que certaines observations restent difficiles à prédire.

5.3.4 KNN (K-Nearest Neighbors)

Le modèle KNN (K-Nearest Neighbors) est une méthode non paramétrique qui consiste à prédire la valeur d'une observation en s'appuyant sur les k observations les plus proches dans l'espace des variables explicatives. La prédiction est obtenue en calculant la moyenne des valeurs des voisins les plus proches.

Contrairement à d'autres modèles, le KNN ne repose pas sur une phase d'apprentissage explicite : il mémorise les données d'entraînement et effectue les prédictions en calculant des distances entre les observations. La qualité du modèle dépend donc fortement du choix de k et de la mesure de distance utilisée.

La Figure 5.15 présente les prédictions du salaire obtenues avec le modèle KNN.

```python
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


model_knn = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsRegressor(n_neighbors=5))
])

model_knn.fit(X_train, y_train)
y_pred_knn = model_knn.predict(X_test)


mse_knn = mean_squared_error(y_test, y_pred_knn)
rmse_knn = np.sqrt(mse_knn)
mae_knn = mean_absolute_error(y_test, y_pred_knn)
r2_knn = r2_score(y_test, y_pred_knn)

print("RMSE :", rmse_knn)
print("MAE :", mae_knn)
print("R2 :", r2_knn)
```

```
RMSE : 16609.88822120727
MAE : 12337.001799999998
R2 : 0.7353565506795872
```

Figure 5.15: Prédictions du salaire avec le KNN

Le modèle KNN obtient un R^2 d'environ 0,74, ce qui signifie qu'il explique une part importante, mais encore limitée, de la variabilité du salaire.

Les valeurs du RMSE et du MAE, respectivement d'environ 16 610 dollars et 12 337 dollars, montrent que les erreurs de prédiction restent relativement élevées. En moyenne, les salaires prédits s'écartent donc de manière assez marquée des salaires réels.

Ces résultats indiquent que le modèle parvient à capter une partie de l'information contenue dans les données, mais que sa précision reste modérée pour ce problème de prédiction.

La Figure 5.16 présente les résultats de la validation croisée.

```python
#Validation croisée et choix du meilleur k
k_values = range(1, 21)
mean_scores = []

for k in k_values:
    model = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsRegressor(n_neighbors=k))
    ])

    scores = cross_val_score(model, X_train, y_train, cv=5, scoring="r2")
    mean_scores.append(scores.mean())

    print(f"k = {k}")
    print("Scores R2 :", scores)
    print("Moyenne :", scores.mean())
    print("-" * 40)
```

```
k = 1
Scores R2 : [0.43805315 0.4728589  0.45463997 0.52985404 0.47685682]
Moyenne : 0.47445257500864846
----------------------------------------
k = 2
Scores R2 : [0.62280519 0.60576784 0.6019479  0.66292087 0.63083697]
Moyenne : 0.6248557560620387
----------------------------------------
```

Figure 5.16: Résultats de la validation croisée pour le KNN

La validation croisée permet d'évaluer l'impact du paramètre k sur les performances du modèle KNN. On observe que pour de petites valeurs de k, les scores R^2 sont faibles, ce qui traduit un modèle peu performant et instable.

Lorsque k augmente, les scores s'améliorent progressivement, ce qui montre que le modèle gagne en stabilité et en capacité de généralisation. Les performances atteignent un maximum autour de $k \approx 14$ ou 15, avec un R^2 moyen d'environ 0,77.

Au-delà de cette valeur, les performances commencent légèrement à diminuer, ce qui indique que le modèle devient trop lissé et perd en précision.

La Figure 5.17 présente les résultats obtenus avec la meilleure valeur de k.

```
Meilleur k : 15
Meilleur score moyen en validation croisée : 0.7686415675918391
Résultats du KNN avec validation croisée :
Best k : 15
RMSE : 14670.073008046915
MAE : 11327.441666666668
R2 : 0.7935607468661848
```

Figure 5.17: Résultats avec le meilleur k

En prenant le meilleur $k = 15$ déterminé par validation croisée, les performances du modèle s'améliorent. Le R^2 atteint environ 0,79, tandis que les erreurs, mesurées par le RMSE et le MAE, diminuent, indiquant une meilleure qualité de prédiction.

Analyse des résidus

La figure ci-dessous présente la distribution des résidus associée au modèle K-Nearest Neighbors.

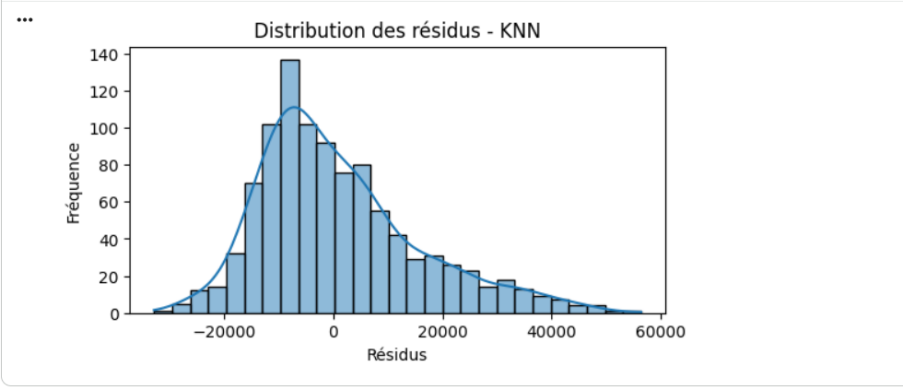

Figure 5.18: Distribution des résidus du KNN

La distribution des résidus du modèle KNN n'est pas parfaitement symétrique et présente une asymétrie positive, indiquant une tendance du modèle à sous-estimer certaines valeurs élevées.

Les résidus ne sont pas strictement centrés autour de zéro, ce qui suggère la présence d'un biais dans les prédictions. De plus, la dispersion importante et la présence de valeurs extrêmes montrent que certaines observations sont difficilement prédites, traduisant une variabilité élevée des erreurs.

Nous avons également représenté les résidus en fonction des valeurs prédites, comme illustré dans la figure ci-dessous :

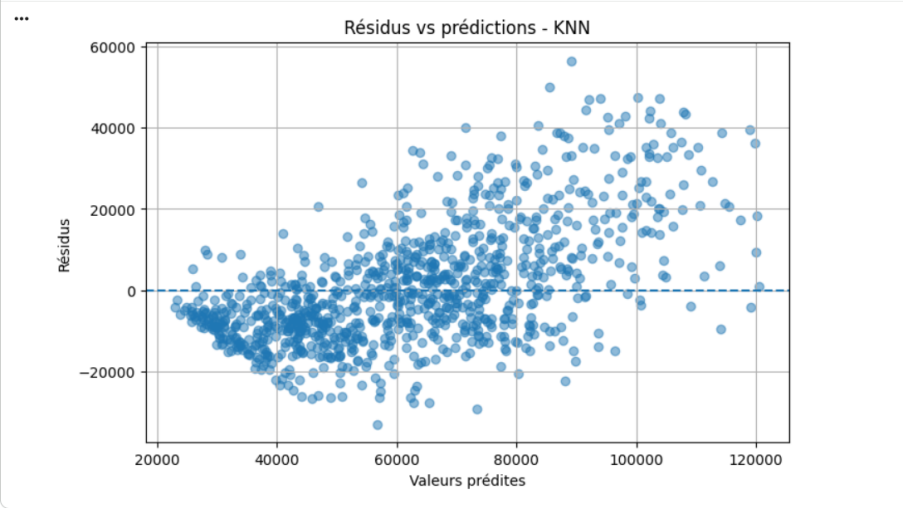

Figure 5.19: Résidus en fonction des valeurs prédites pour le KNN

On observe que les résidus ne sont pas répartis de manière aléatoire autour de zéro, mais présentent une structure croissante. En effet, plus les valeurs prédites augmentent, plus la dispersion des résidus devient importante.

Ce phénomène indique que le modèle commet des erreurs de plus en plus élevées pour les salaires importants. La présence d'une structure dans les résidus suggère également que le modèle ne capture pas correctement certaines relations dans les données, ce qui traduit une qualité de prédiction limitée.

5.3.5 Réseau de neurones

Le réseau de neurones est un modèle d'apprentissage supervisé composé de plusieurs couches de neurones artificiels connectés entre eux. Chaque neurone transforme les informations reçues puis les transmet à la couche suivante. Grâce aux fonctions d'activation, le modèle peut apprendre des relations complexes et non linéaires entre les variables.

Pendant l'apprentissage, le réseau ajuste progressivement ses poids afin de réduire l'écart entre les valeurs prédites et les valeurs réelles. Cette correction repose sur la descente de gradient et la rétropropagation. Ce modèle est particulièrement intéressant pour traiter des problèmes où les relations entre les variables sont complexes, même s'il reste plus difficile à interpréter que des modèles plus classiques.

Les Figures 5.20 et 5.21 présentent les prédictions du salaire obtenues avec le réseau de neurones.

```python
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.pipeline import Pipeline

# Pipeline : normalisation + modèle
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', MLPRegressor(
        hidden_layer_sizes=(100, 50),
        activation='relu',
        solver='adam',
        max_iter=500,
        random_state=42
    ))
])


# Entraînement final sur tout le train set
pipeline.fit(X_train, y_train)

# Prédictions sur le test set
y_pred_nn = pipeline.predict(X_test)
```

Figure 5.20: Prédictions du salaire avec le réseau de neurones

```python
# Métriques
mse_nn = mean_squared_error(y_test, y_pred_nn)
rmse_nn = np.sqrt(mse_nn)
mae_nn = mean_absolute_error(y_test, y_pred_nn)
r2_nn = r2_score(y_test, y_pred_nn)

print("\n--- Évaluation finale ---")
print("RMSE:", rmse_nn)
print("MAE :", mae_nn)
print("R2  :", r2_nn)
```

```
...
--- Évaluation finale ---
RMSE: 7387.45927927981
MAE : 5458.3984367063695
R2  : 0.9476498501915073
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_pe
  warnings.warn(
```

Figure 5.21: Prédictions du salaire avec le réseau de neurones (suite)

Le modèle de réseau de neurones présente de bonnes performances, avec un R^2 d'environ 0,95, ce qui indique qu'il explique une grande partie de la variabilité du salaire.

Les valeurs du RMSE et du MAE, respectivement d'environ 7 387 dollars et 5 458 dollars, montrent que les erreurs de prédiction restent modérées. En moyenne, les salaires prédits sont relativement proches des salaires réels, ce qui traduit une bonne capacité du modèle à capturer les relations présentes dans les données.

Ces résultats suggèrent que le réseau de neurones est capable de modéliser des relations complexes entre les variables explicatives et le salaire, tout en conservant une bonne qualité de prédiction.

La Figure 5.22 présente les résultats de la validation croisée.

```python
# Cross-validation sur les données d'entraînement uniquement
cv_scores = cross_val_score(pipeline, X_train, y_train, cv=5, scoring='r2')
print("Scores R2 (CV) :", cv_scores)
print("Score moyen CV :", cv_scores.mean())
```

```
...
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning:
  warnings.warn(
Scores R2 (CV) : [0.94286236 0.94369694 0.94408604 0.94543066 0.94497937]
Score moyen CV : 0.9442110760288639
```

Figure 5.22: Résultats de la validation croisée pour le réseau de neurones

Les scores obtenus en validation croisée sont très proches les uns des autres et se situent autour de 0,94. Cela montre que le modèle explique une part importante de la variabilité des salaires, tout en conservant une bonne stabilité d'un pli à l'autre.

La faible dispersion des scores suggère donc une capacité de généralisation satisfaisante sur les données d'entraînement.

On observe toutefois un avertissement de convergence, ce qui signifie que le réseau de neurones n'a pas totalement convergé dans le nombre d'itérations fixé.

Malgré cela, les performances obtenues demeurent satisfaisantes, ce qui montre que le modèle reste globalement efficace pour ce problème de prédiction.

Analyse des résidus

La figure ci-dessous présente la distribution des résidus du modèle de réseau de neurones.

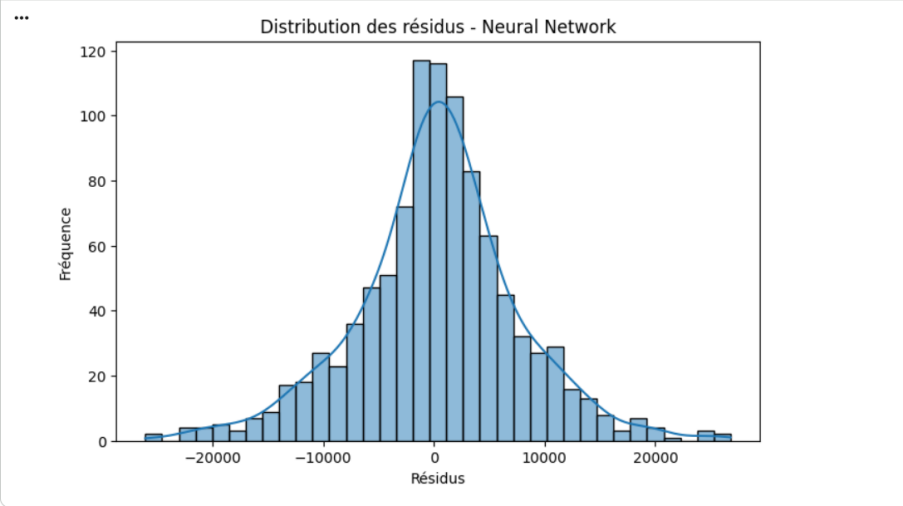

Figure 5.23: Distribution des résidus du réseau de neurones

On observe que les résidus sont globalement centrés autour de zéro et présentent une forme relativement symétrique, proche d'une distribution normale. Cela indique que les erreurs sont réparties de manière aléatoire et que le modèle ne présente pas de biais systématique important.

La majorité des résidus étant concentrée autour de zéro, cela traduit une bonne qualité globale de prédiction. On note toutefois la présence de quelques valeurs extrêmes, ce qui montre que certaines observations restent plus difficiles à prédire.

Nous avons également représenté les résidus en fonction des valeurs prédites, comme illustré dans la figure ci-dessous :

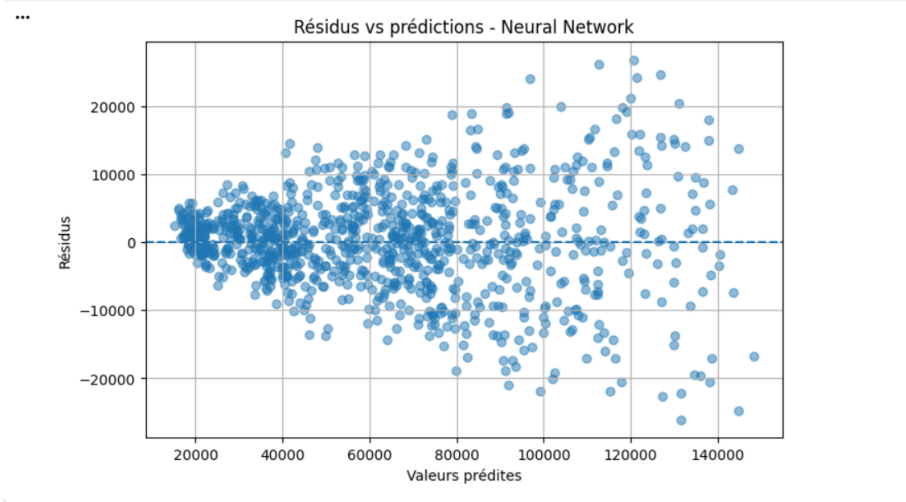

Figure 5.24: Résidus en fonction des valeurs prédites pour le réseau de neurones

On observe que les résidus sont globalement répartis de manière aléatoire autour de zéro, ce qui indique que le modèle ne présente pas de biais systématique important.

Cependant, la dispersion des résidus tend à augmenter pour les valeurs prédites les plus élevées, ce qui suggère que le modèle est moins précis pour les salaires élevés. Malgré cela, l'absence de structure marquée dans la répartition des erreurs confirme une bonne cohérence globale du modèle.

5.3.6 Comparaison des modèles

La Figure 6.5 présente une synthèse des performances des différents modèles étudiés.

Model	RMSE	MAE	R2
Gradient Boosting	6395.021673	4913.543777	0.960771
Random Forest	6795.105116	5139.035510	0.955709
Neural Network	7387.459279	5458.398437	0.947650
Linear Regression	9496.938007	7406.001936	0.913484
KNN	14670.073008	11327.441667	0.793561

Figure 5.25: Comparaison des performances des modèles

L'étude comparative des modèles de régression montre que les méthodes basées sur les arbres de décision, en particulier le Gradient Boosting et le Random Forest, offrent les meilleures performances, avec des coefficients de détermination supérieurs à 0,95 et des erreurs (RMSE et MAE) plus faibles.

Ces modèles sont capables de capturer des relations non linéaires complexes entre les variables explicatives et le salaire, ce qui explique leur supériorité par rapport aux modèles plus simples.

Le réseau de neurones présente également de très bonnes performances, bien que légèrement inférieures à celles des modèles d'arbres. Cette différence peut s'expliquer par la sensibilité du modèle aux paramètres et aux conditions d'optimisation.

La régression linéaire, bien que plus simple et facilement interprétable, montre des performances correctes mais limitées. Cela suggère que la relation entre les variables explicatives et le salaire n'est pas strictement linéaire.

Enfin, le modèle KNN apparaît comme le moins performant, avec des erreurs plus élevées, indiquant qu'il est moins adapté à ce type de données.

L'analyse des résidus met en évidence un phénomène d'hétéroscédasticité, les erreurs ayant tendance à augmenter avec les valeurs de salaire, ainsi que la présence de valeurs extrêmes. Cela indique que les salaires élevés sont plus difficiles à prédire.

Malgré ces limites, les modèles les plus performants, en particulier le Gradient Boosting, restent globalement fiables et précis. Ce modèle apparaît ainsi comme le plus adapté pour la prédiction du salaire.

Ces résultats confirment l'importance du choix du modèle dans un problème de prédiction, en particulier lorsque les relations entre les variables sont complexes et non linéaires.

Chapitre 6

Description technique et fonctionnelle

6.1 Introduction

Dans le cadre de ce projet, nous avons développé une application web nommée *SalaryPredict*, dont l'objectif est d'estimer le salaire d'une offre d'emploie.

Le site repose sur l'utilisation de modèles de machine learning entraînés sur un ensemble de données réelles. À partir des informations renseignées par l'utilisateur (région, secteur, niveau de séniorité, intensité de l'intelligence artificielle, etc.).

Au-delà de la simple prédiction, le site propose également des fonctionnalités d'analyse et de comparaison, permettant de mieux comprendre les facteurs influençant les rémunérations.

Ainsi, *SalaryPredict* constitue à la fois un outil de prédiction et un support d'analyse, offrant une vision globale des mécanismes influençant les salaires à partir de données réelles.

6.2 Description technique

Technologies utilisées

- **Back-end :** PHP, Python, Flask, API REST, cURL
- **Front-end :** HTML, CSS, JavaScript
- **Base de données :** MySQL, Php Myadmin
- **Machine learning :** scikit-learn
- **Visualisation et export :** Chart.js, jsPDF
- **Partage et suivi du code :** GitHub
- **Utilisation d'outils d'assistance :** Des outils d'intelligence artificielle générative ont été utilisés de manière ponctuelle comme aide à la relecture et au débogage. Claude a notamment été mobilisé pour assister la correction de certains bugs dans le code, tandis que ChatGPT a été utilisé pour améliorer la formulation de certaines phrases et corriger des fautes dans le rapport.

Schéma de l'architecture du site web

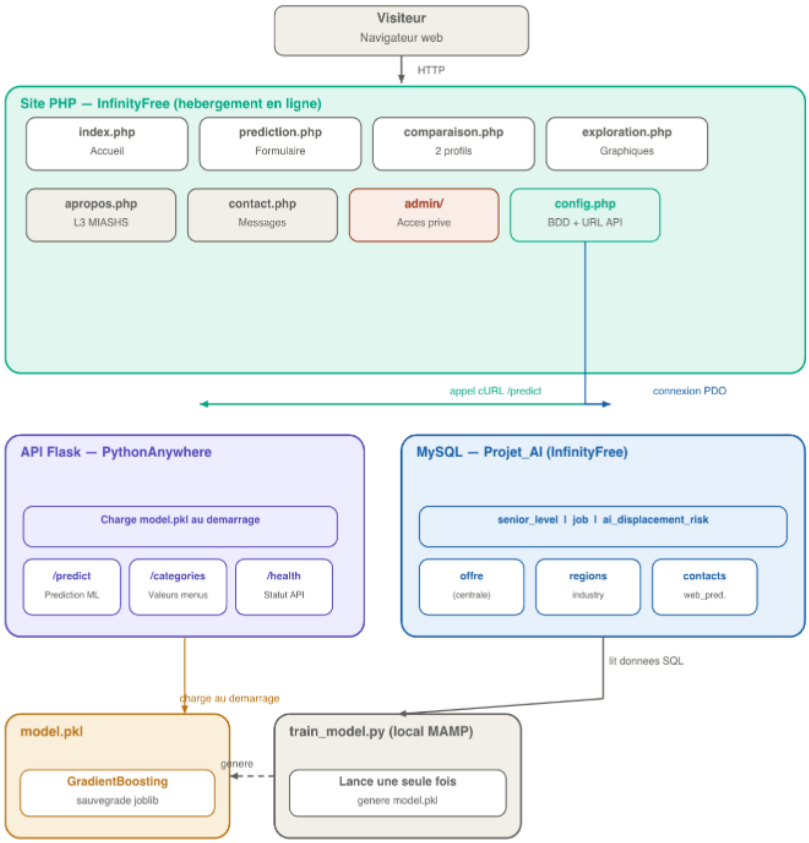

Figure 6.1: Schéma de l'architecture du site web

6.3 Description fonctionnelle

6.3.1 Page d'accueil

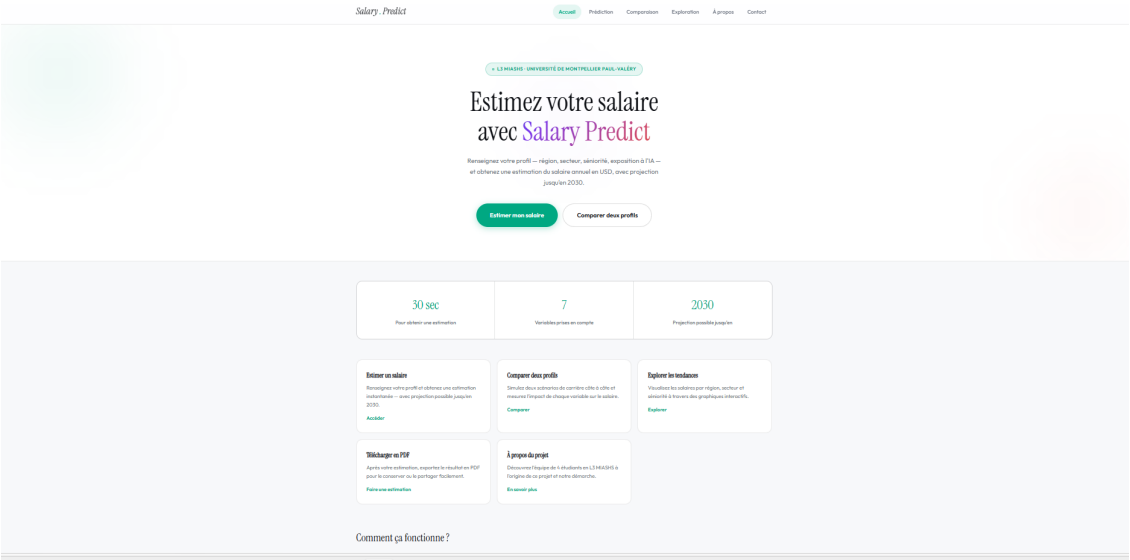

Figure 6.2: Page index.php

La page d'accueil constitue le point d'entrée principal de l'application *Salary Predict*. Elle permet de présenter rapidement l'objectif du site, orienter l'utilisateur vers les fonctionnalités principales et rendre la navigation intuitive.

Plusieurs fonctionnalités sont accessibles depuis cette page :

- **Accéder à la prédiction de salaire** : l'utilisateur peut cliquer sur le bouton *« Estimer mon salaire »* pour être redirigé vers la page de prédiction, où il pourra prédire un salaire.
- **Comparer deux profils** : un second bouton permet d'accéder à une fonctionnalité de comparaison entre deux profils afin d'observer l'impact de certaines variables sur le salaire estimé.
- **Explorer les tendances** : la page d'accueil donne également accès à une section d'exploration des données, permettant de visualiser les salaires selon différents critères à travers des graphiques.
- **Télécharger un résultat en PDF** : l'utilisateur est informé qu'après avoir effectué une estimation, il pourra exporter le résultat obtenu sous format PDF.
- **Consulter les informations sur le projet** : une carte dédiée permet d'accéder à la page *À propos*.

La page d'accueil joue également un rôle informatif. Des indicateurs visuels mettent en avant certaines caractéristiques du service, comme la rapidité d'obtention d'une estimation, le nombre de variables prises en compte et la possibilité d'effectuer une projection jusqu'en 2030. Une section *« Comment ça fonctionne ? »* explique enfin les différentes étapes suivies par l'utilisateur, depuis le remplissage du formulaire jusqu'à l'export du résultat.

Concernant la gestion des erreurs, la page d'accueil ne comporte pas de saisie directe de données.

6.3.2 Page À propos

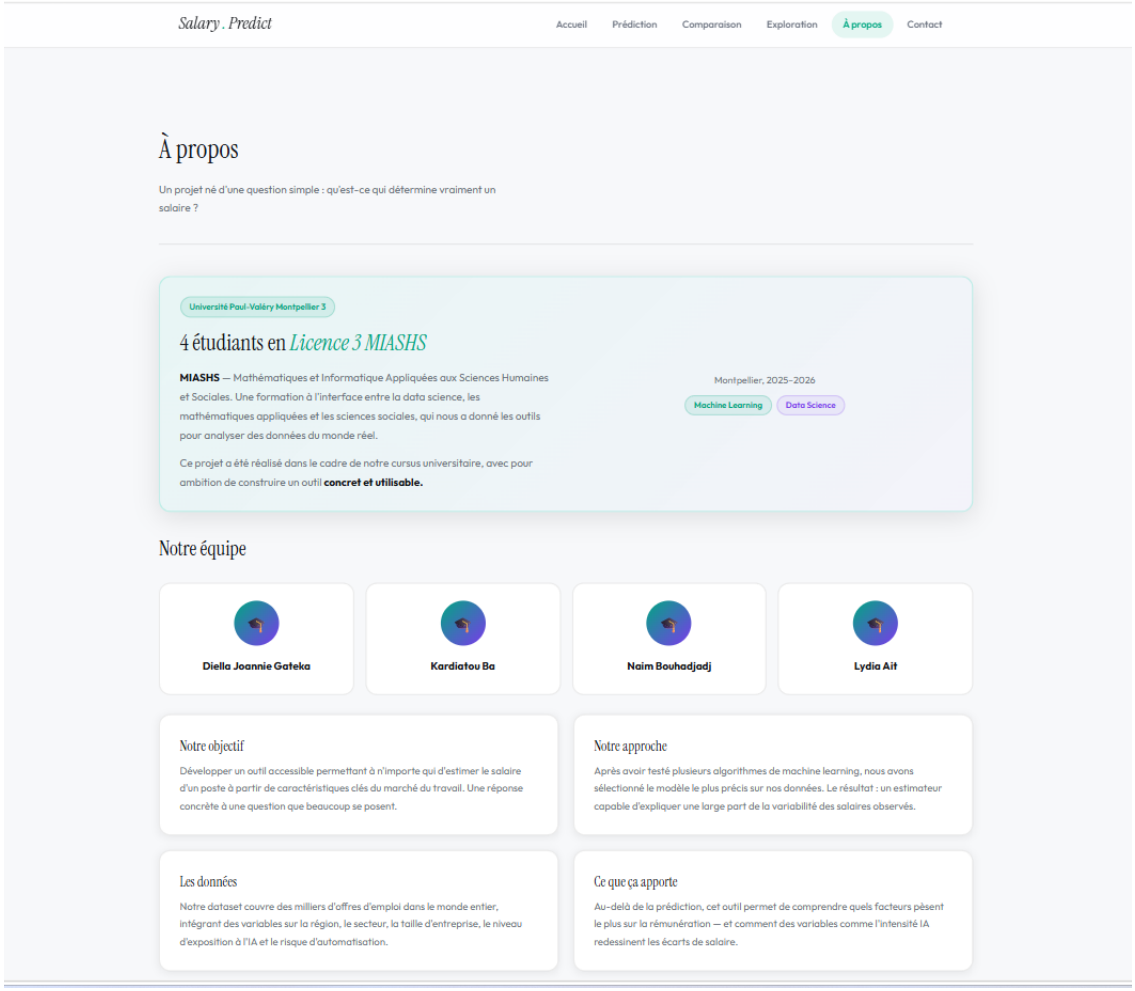

Figure 6.3: Page À propos

La page *À propos* permet de présenter le contexte du projet, l'équipe ayant développé l'application ainsi que les objectifs poursuivis. Elle a avant tout une fonction informative et permet à l'utilisateur de mieux comprendre l'intérêt du site.

L'utilisateur peut y trouver :

- une présentation des membres de l'équipe ;
- une explication de l'objectif de l'application ;
- un résumé de l'approche retenue en machine learning ;
- une description synthétique des données exploitées ;

Cette page ne comporte pas de saisie directe, ce qui limite fortement les risques d'erreur de la part de l'utilisateur.

6.3.3 Page de prédiction

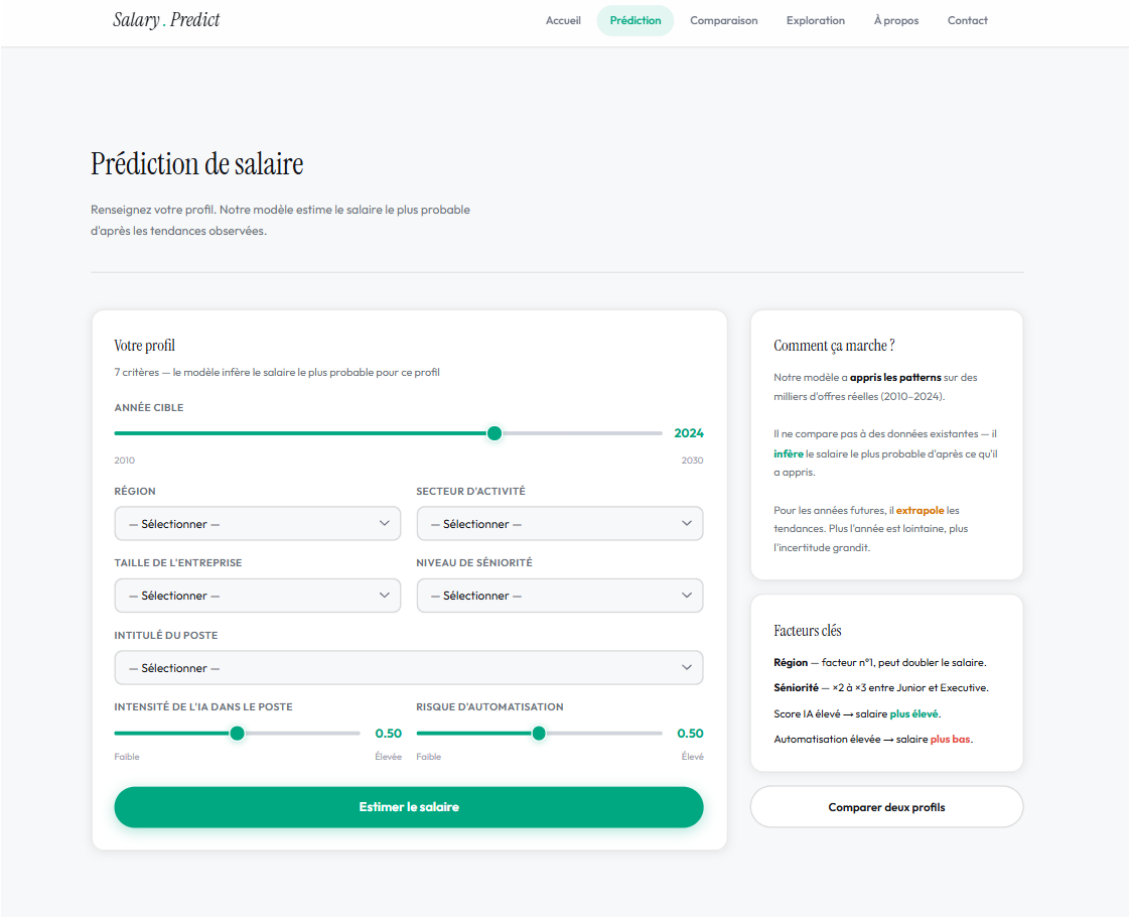

Figure 6.4: Page prediction.php

La page de prédiction permet à l'utilisateur de prédire un salaire à partir de plusieurs variables, telles que l'année, la région, le secteur d'activité, la taille de l'entreprise, le niveau de séniorité, l'intitulé du poste, l'intensité de l'IA et le risque d'automatisation.

L'utilisateur peut ainsi :

- compléter un formulaire guidé pour prédire un salaire annuel;
- lire une interprétation du résultat ;

Plusieurs mécanismes de gestion des erreurs ont été prévus :

- contrôle des champs obligatoires ;
- blocage des années en dehors de l'intervalle 2010-2030 ;
- affichage d'un message d'erreur si le formulaire est incomplet ;
- avertissement en cas de projection sur des années futures car notre jeu de données s'arrête à l'année 2025 ;
- désactivation temporaire du bouton lors du traitement pour éviter les doubles soumissions.

6.3.4 Page de comparaison

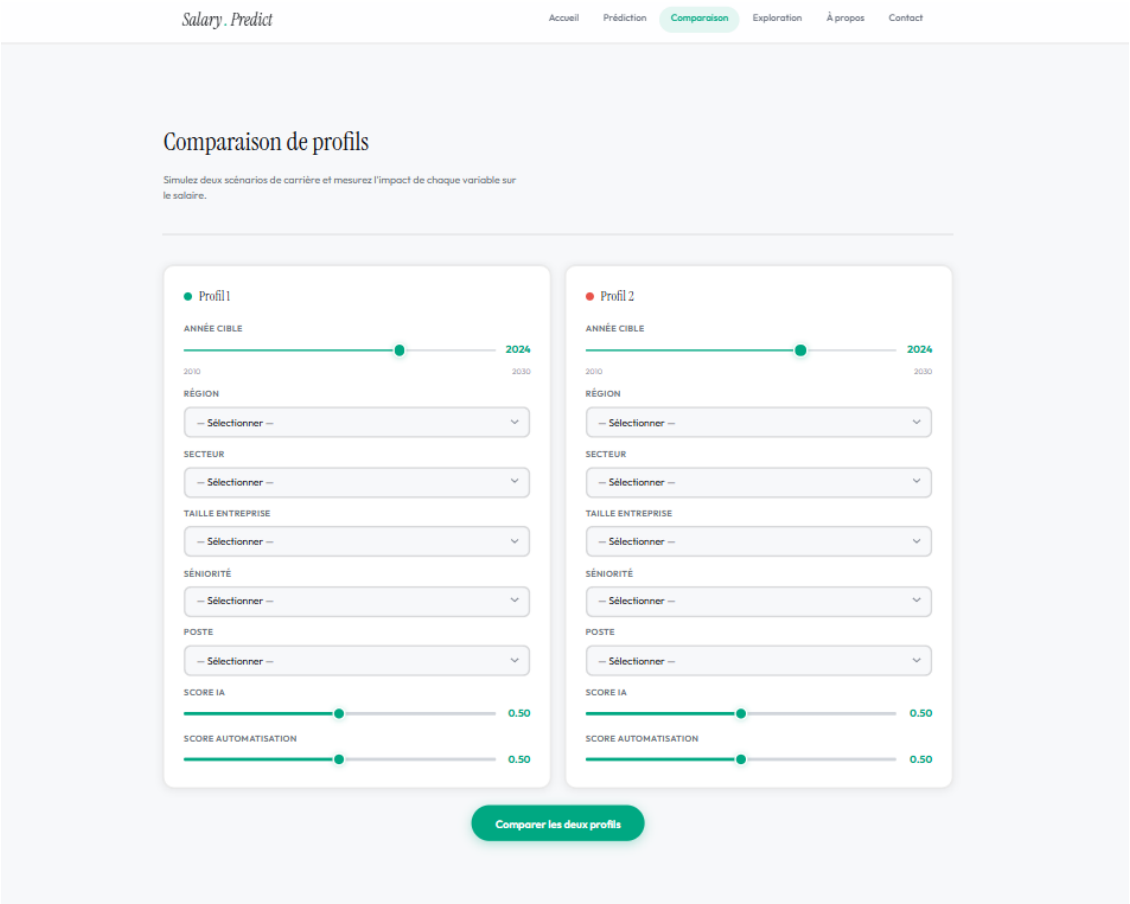

Figure 6.5: Page comparaison.php

La page de comparaison permet à l'utilisateur de renseigner deux profils professionnels distincts afin d'observer les différences de salaire estimé entre eux. Elle reprend les mêmes variables que la page de prédiction, mais applique le calcul à deux profils en parallèle.

L'utilisateur peut ainsi :

- saisir deux profils différents ;
- obtenir une estimation de salaire pour chacun ;
- mesurer l'écart de rémunération en valeur absolue et en pourcentage ;
- identifier quel profil est le mieux rémunéré ;
- visualiser les résultats sous forme de barres comparatives.

Concernant la gestion des erreurs, les éléments suivants ont été prévus :

- contrôle de la complétude des champs des deux profils ;
- affichage d'un message d'erreur si un champ manque ;

6.3.5 *Page d'exploration des données*

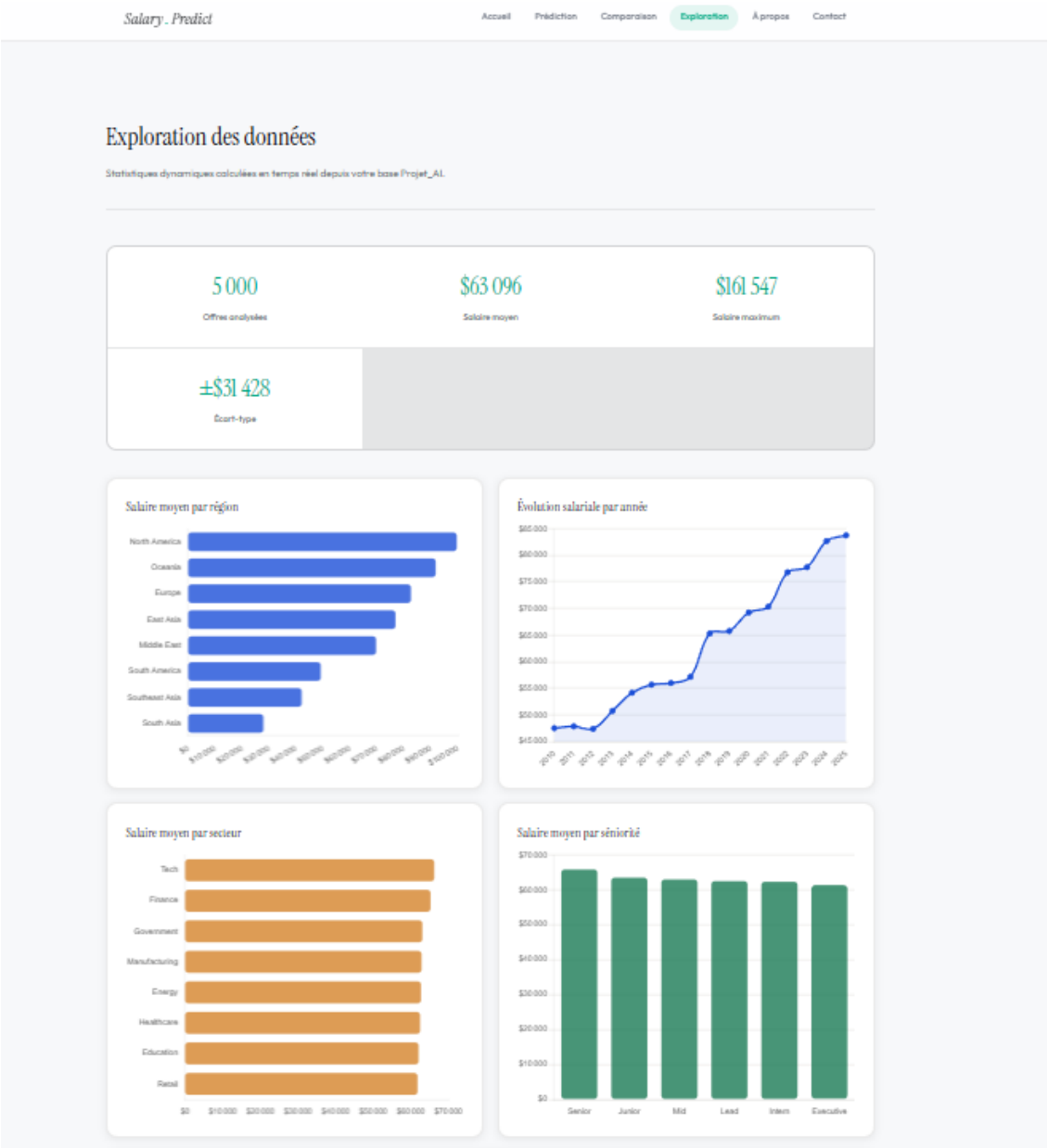

Figure 6.6: Page d'exploration des données

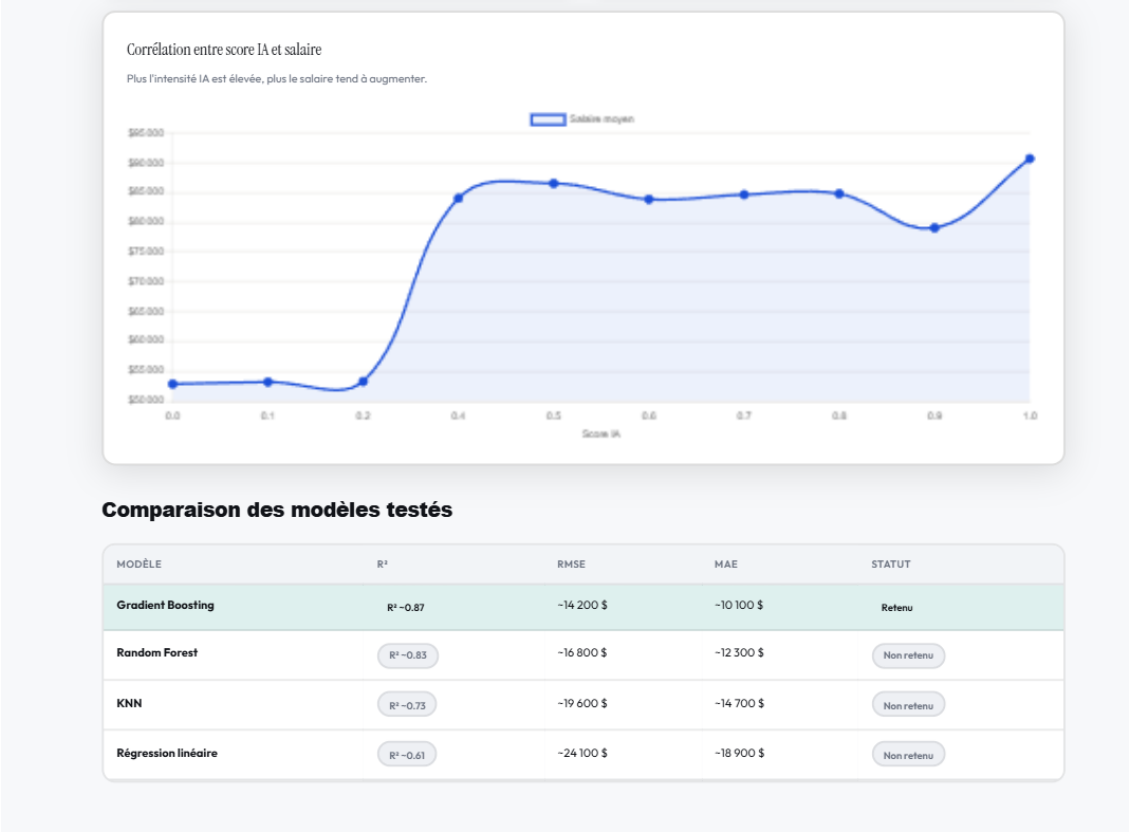

Figure 6.7: Visualisations complémentaires et comparaison des modèles

La page d'exploration des données permet à l'utilisateur de visualiser les principales tendances du jeu de données utilisé dans le projet. Elle affiche des statistiques globales, comme le nombre d'offres, le salaire moyen, le salaire maximum et l'écart-type.

L'utilisateur peut également consulter plusieurs graphiques, notamment le salaire moyen par région, par secteur, par niveau de séniorité, l'évolution des salaires selon les années, ainsi que la relation entre l'intensité de l'intelligence artificielle et le salaire.

Concernant la gestion des erreurs, cette page dépend de la disponibilité de la base de données et du bon chargement de la bibliothèque *Chart.js*. En cas d'absence de données ou d'échec d'une requête, certaines statistiques peuvent être nulles ou certains graphiques peuvent ne pas s'afficher correctement.

6.3.6 *Page de contact*

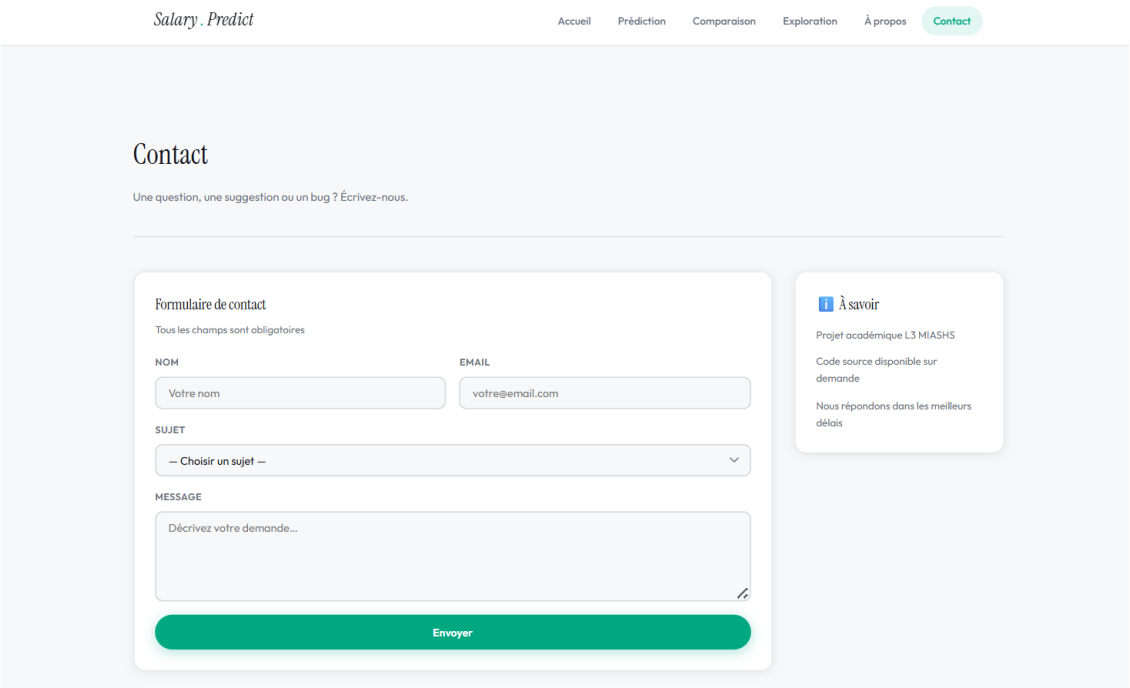

Figure 6.8: Page contact.php

La page de contact permet à l'utilisateur d'envoyer un message à l'équipe du projet en cas de question, de suggestion ou de signalement de bug. Elle comporte un formulaire avec quatre champs : le nom, l'adresse email, le sujet et le message.

Cette page permet donc :

- de contacter l'équipe du projet ;
- de choisir un sujet parmi plusieurs catégories ;
- d'envoyer un message directement depuis l'interface ;
- d'afficher un message de confirmation après envoi.

Concernant la gestion des erreurs, tous les champs sont obligatoires. Si un champ est vide, un message d'erreur est affiché. L'adresse email est également vérifiée afin de s'assurer qu'elle respecte un format valide. Enfin, si la base de données n'est pas disponible ou si l'enregistrement échoue, un message d'erreur informe l'utilisateur que le message n'a pas pu être envoyé.

6.3.7 *Espace administrateur*

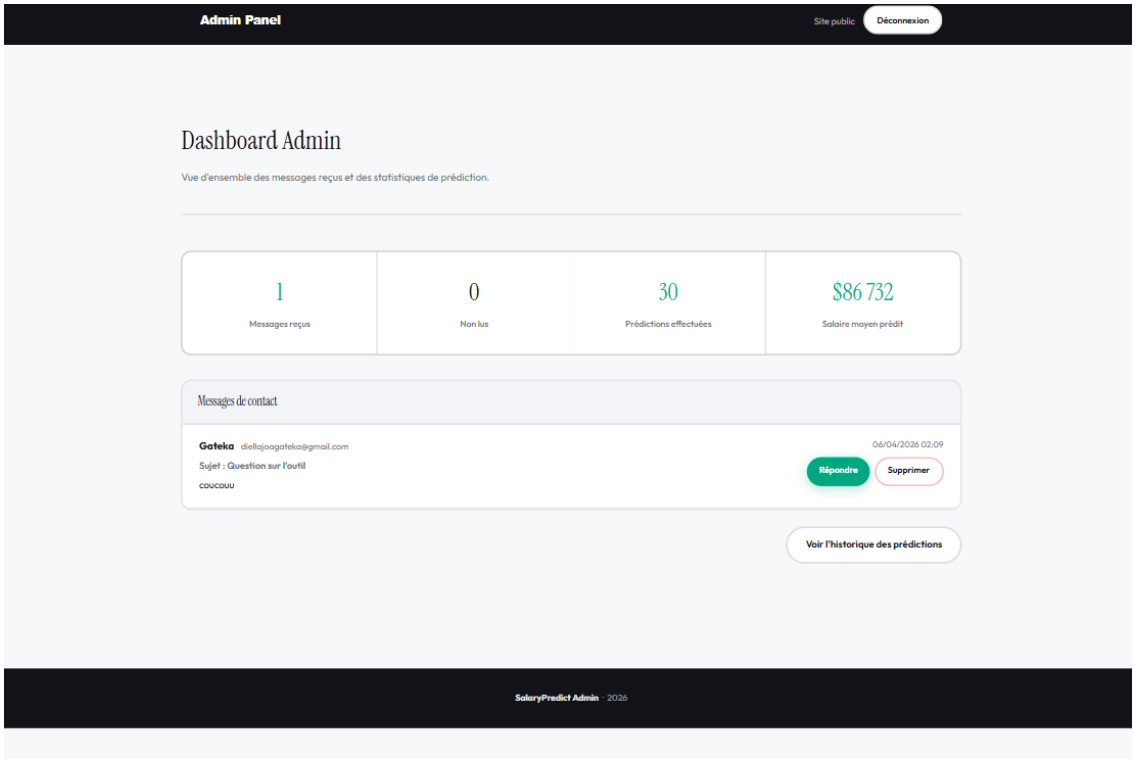

Figure 6.9: page index.php des administrarteurs

L'espace administrateur permet de centraliser la gestion des messages reçus via le formulaire de contact ainsi que le suivi de certaines statistiques du site. Il est protégé par une authentification avec identifiant et mot de passe. Et elle accessible en cliquant sur le lien se trouvant sur le footer de la page d'accueil.

Cette page permet :

- de lire les messages envoyés par les utilisateurs ;
- de répondre directement par email ;
- de supprimer un message ;
- d'accéder à l'historique des prédictions.

Concernant la gestion des erreurs, l'accès à cette page est limité aux utilisateurs authentifiés. En cas d'identifiants incorrects, un message d'erreur est affiché. De plus, certaines actions, comme la suppression d'un message, sont sécurisées par une demande de confirmation. Enfin, si la base de données n'est pas accessible, les statistiques ou les messages peuvent ne pas s'afficher correctement.

6.3.8 *Page d'historique des prédictions*

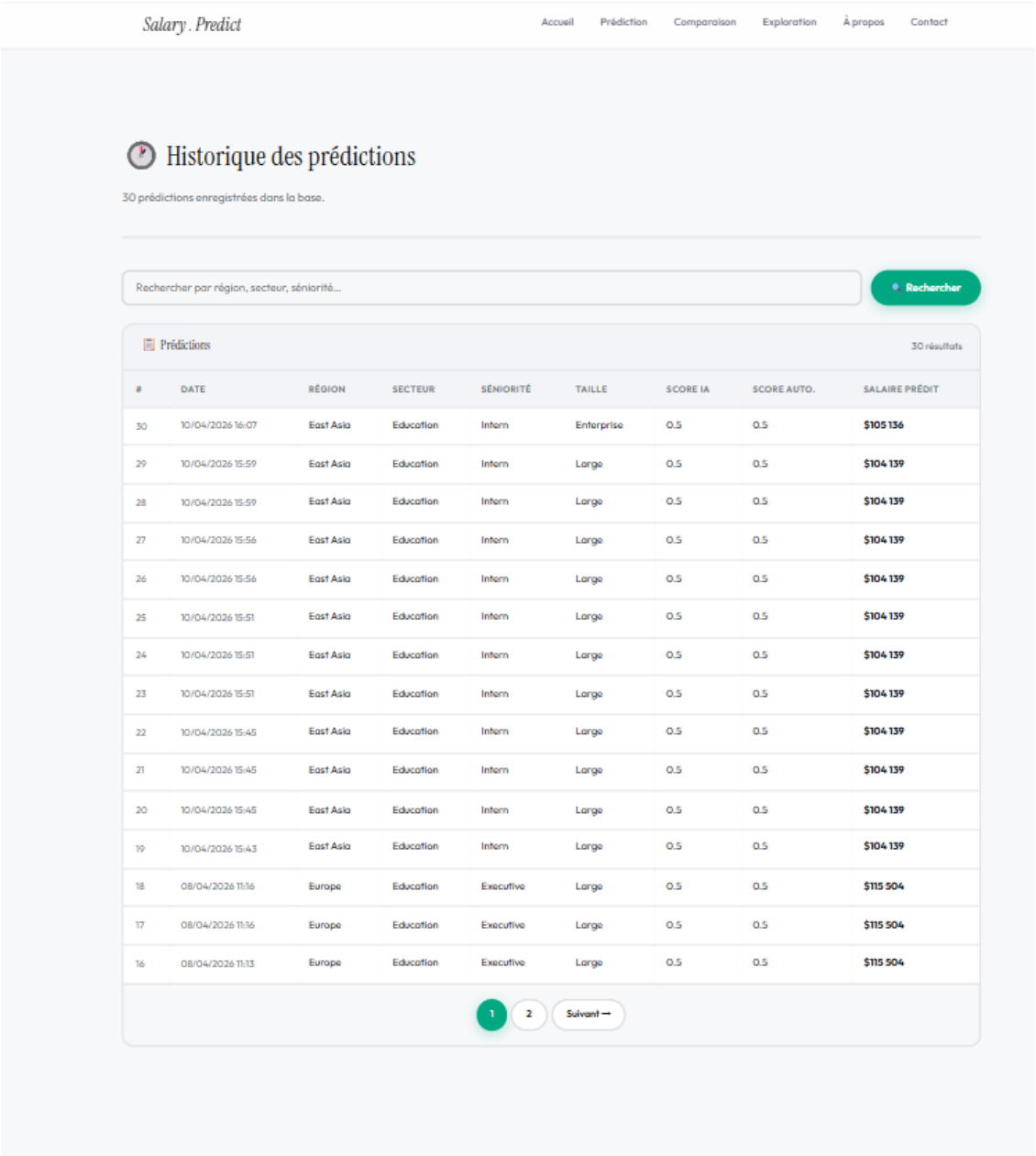

Figure 6.10: Historique des prédictions

La page d'historique des prédictions permet de consulter l'ensemble des estimations enregistrées dans la base de données.

Cette page permet :

- d'accéder rapidement à un historique structuré des estimations produites par le site.

Concernant la gestion des erreurs,cette page dépend de la disponibilité de la base de données : en cas de problème de connexion, les données peuvent ne pas s'afficher correctement.

6.4 Conclusions

Ce projet nous a permis de développer une application complète de prédiction de salaires, en mobilisant à la fois des compétences en data science et en développement web. À partir d'un jeu de données structuré, nous avons pu analyser les facteurs influençant les salaires, entraîner plusieurs modèles de machine learning et intégrer le modèle le plus performant dans une interface web interactive.

L'utilisation du Gradient Boosting s'est révélée pertinente, offrant de bonnes performances de prédiction et permettant de capter des relations complexes entre les variables. Le site développé permet ainsi à l'utilisateur d'obtenir une estimation de salaire tout en visualisant les tendances générales des données.

Difficultés rencontrées :

- La gestion de la base de données a posé problème, notamment avec le modèle organisationnel des données (MOD) : lors de l'importation, les clés étrangères pouvaient disparaître en les envoyant aux autres membres du groupe, obligeant chaque membre à les recréer manuellement pour assurer la cohérence du site.
- Le jeu de données utilisé présentait des limites : nous avons eu des difficultés à trouver une seconde base compatible afin d'enrichir l'étude avec davantage de variables pertinentes et un plus grand nombre d'intitulés de postes.
- Les données étant limitées dans le temps, jusqu'à l'année 2025, cela réduit la précision des prédictions pour les années récentes et futures. Nous avons néanmoins proposé une approximation basée sur les tendances observées jusqu'en 2030.
- L'hébergement et le déploiement ont également constitué une difficulté importante. Nous avons dû comprendre que le site PHP et l'API Python ne pouvaient pas cohabiter sur le même hébergeur, puis mettre en place une architecture répartie sur deux plateformes distinctes (InfinityFree et PythonAnywhere) communiquant via HTTP.

Malgré ces contraintes, ce projet constitue une expérience enrichissante, nous ayant permis de mieux comprendre les différentes étapes d'un projet en science des données, de la préparation des données jusqu'au déploiement d'une application fonctionnelle.

Chapitre 7

Conclusions et perspectives

7.0.1 Contributions du groupe

Au début du projet, nous avons réparti les tâches afin d'organiser efficacement le travail.

- **Kardiatou** : nettoyage des données, exploration statistique et mise en place des modèles de machine learning pour la prédiction.
- **Joannie** : conception de la base de données, réalisation du MCD et MOD ainsi que développement d'une grande partie du site web, notamment les pages principales (prédiction, comparaison) et les fonctionnalités associées.
- **Naim** : développement de la première version du site web et réalisation de la vidéo de présentation du site.
- **Lydia** : conception du design via Canva.
- **Travail collaboratif** : chaque membre a contribué à certaines pages du site. Le rapport a été rédigé sur Google Docs par Joannie et Kardiatou, puis mis en forme en LaTeX par Kardiatou.

7.0.2 Perspectives

À court terme :

- Ajouter de nouvelles variables pertinentes (niveau d'études, type de contrat, compétences techniques détaillées) afin d'améliorer la précision des prédictions à partir d'un nouveau jeu de données.
- Enrichir davantage les données pour réduire le bruit et améliorer la qualité globale du modèle.
- Améliorer l'interface utilisateur pour rendre le site plus intuitif et ergonomique.
- Ajouter une fonctionnalité de visualisation personnalisée permettant à l'utilisateur de comparer son profil avec d'autres profils similaires.

À long terme :

- Intégrer des données en temps réel (API d'offres d'emploi) afin de rendre les prédictions plus proches du marché actuel.
- Développer un système de recommandation de carrière (suggestion de métiers ou compétences à acquérir pour augmenter son salaire).
- Mettre en place des modèles plus avancés pour améliorer la précision.
- Ajouter une dimension géographique plus fine (ville, pays) pour affiner les prédictions.
- Proposer une version professionnelle du site destinée aux entreprises pour analyser les tendances salariales.

Ces perspectives permettraient de rendre l'application réellement utile dans un contexte professionnel, en aidant à la prise de décision aussi bien pour les candidats que pour les recruteurs.

Annexes

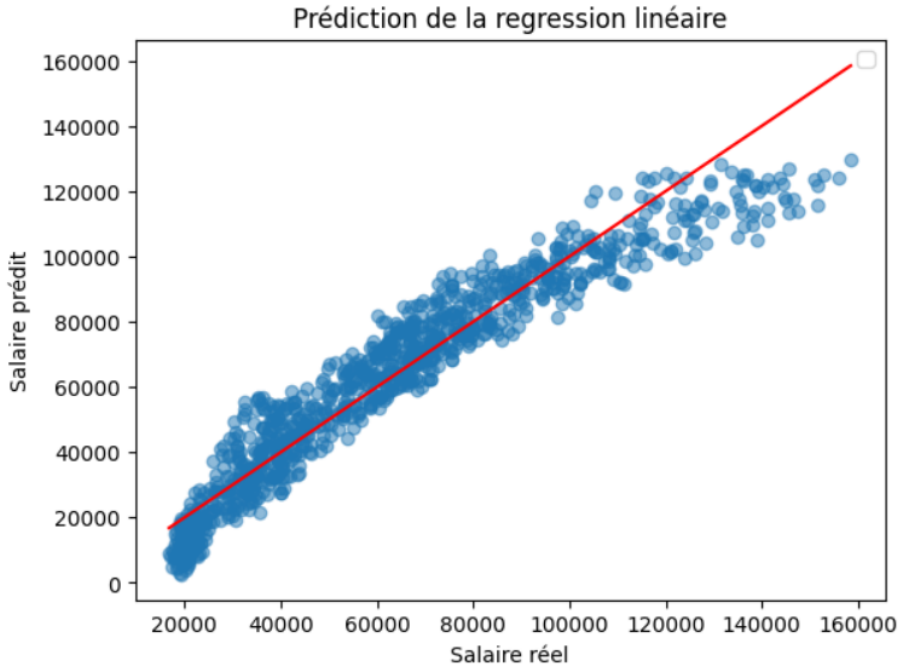

Figure 7.1: Graphique de prédiction avec la régression linéaire

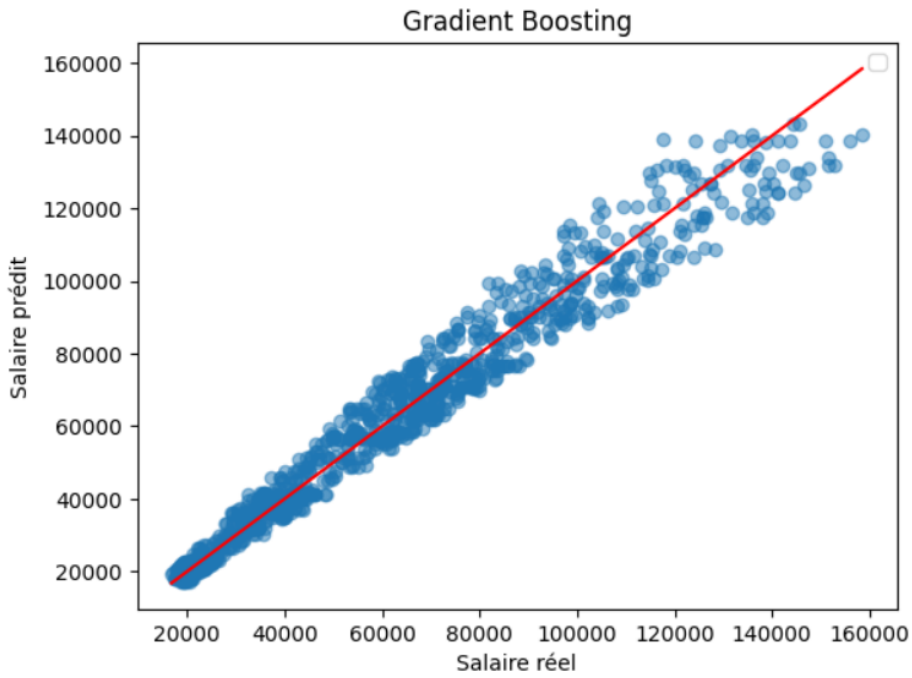

Figure 7.2: Graphique de prédiction avec le Gradient Boosting

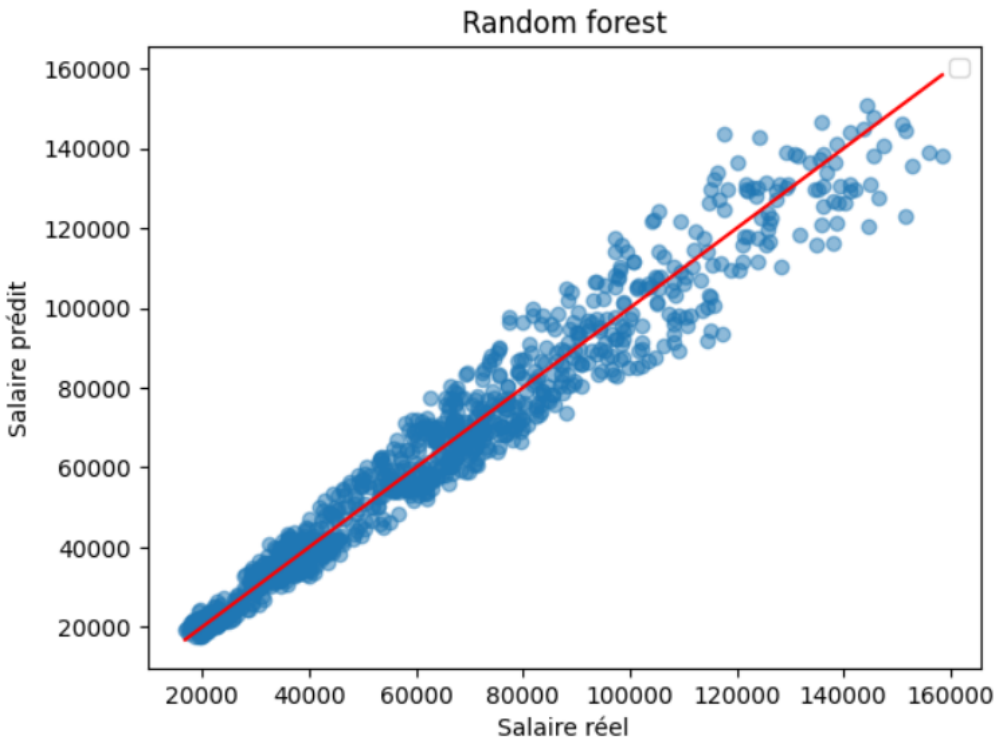

Figure 7.3: Graphique de prédiction avec le Random Forest

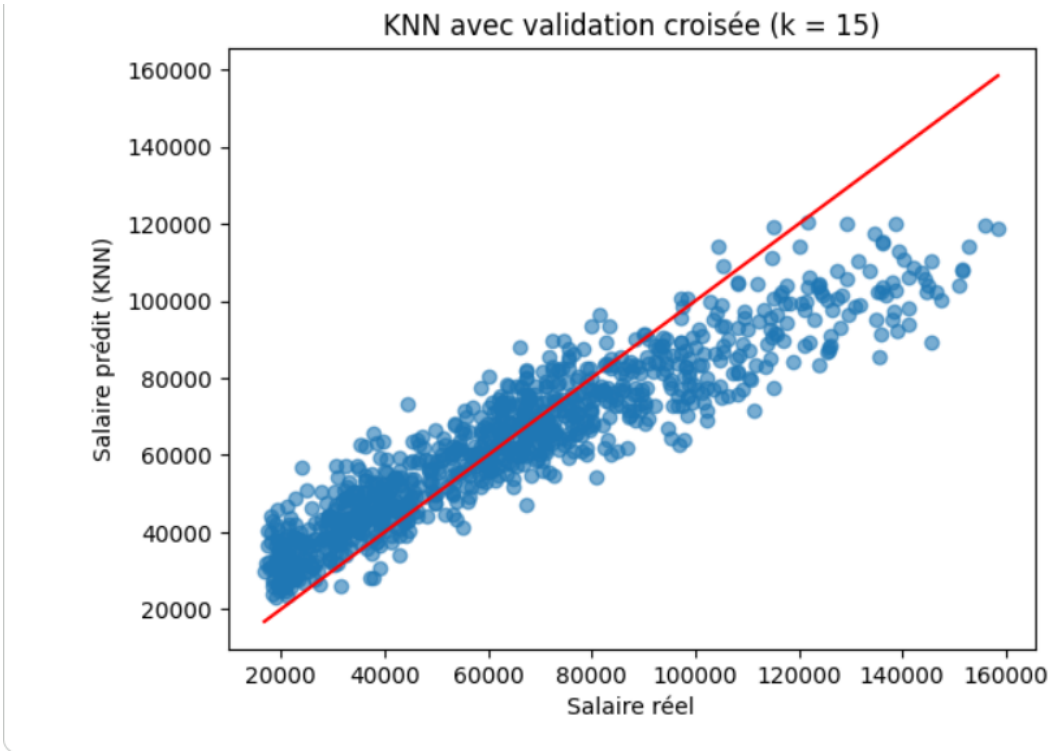

Figure 7.4: Graphique de prédiction avec le KNN

...

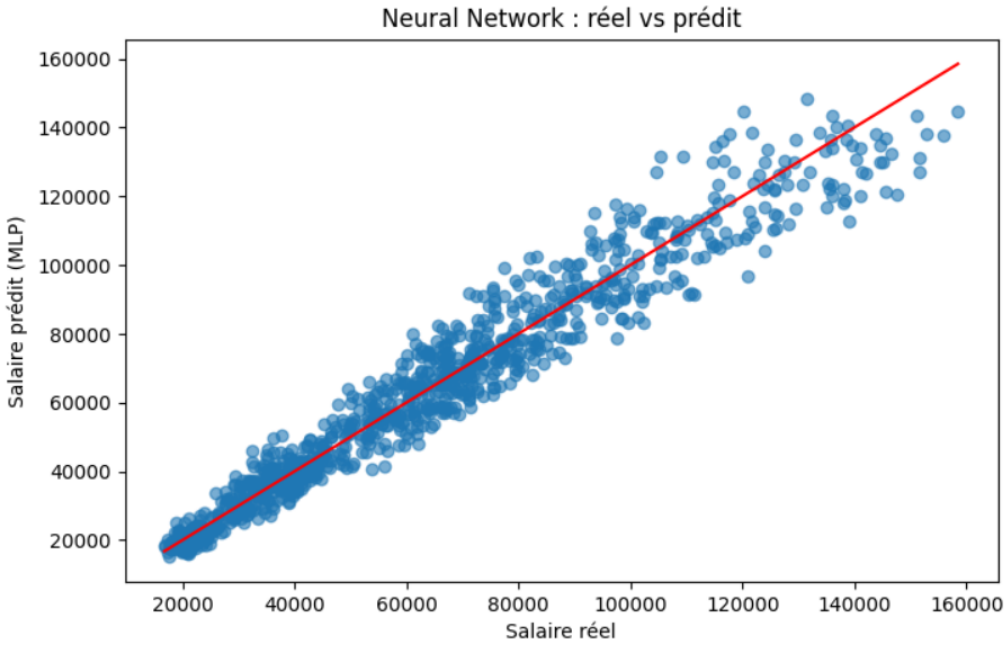

Figure 7.5: Graphique de prédiction avec le réseau de neurones